
Channel Aware Authentication Protocol for Secure Underwater Communication Systems

Overview of Previous Work
and Current Upgrades

Abstract

Underwater wireless communication networks carry environmental monitoring, offshore exploration and naval surveillance traffic, yet they cannot use any of the security machinery that terrestrial networks rely on. Radio does not propagate in seawater, so these networks talk in sound: about 1500 m/s, roughly 10 kbps of usable bandwidth, 10–30 % packet loss, and battery-powered nodes that nobody can service. A TLS handshake on such a link would take tens of seconds and a measurable fraction of a node’s lifetime energy.

This report studies the authentication protocol proposed by Rupa *et al.* in the *IEEE Internet of Things Journal* (Nov. 2025), which is designed specifically for that regime. The protocol arranges the network as a five-layer chain — underwater sensor, submarine, buoy, satellite, base station — and authenticates *each hop independently* using elliptic-curve public-key encryption, a fresh nonce and a bounded timestamp window, completing the seabed-to-shore chain in twelve messages and 2112 bits. It adds dynamic fallback peers so that a failed buoy does not tear down the session, and it is validated with BAN logic and the Scyther model checker.

We reproduce the protocol as an eight-node Python simulator, then strengthen it in seven places where our own first version was unrealistic or unsafe. The most consequential change is cryptographic: our initial build used a hand-written repeating-key XOR cipher, which is broken by a known-plaintext or key-reuse attack in seconds; we replaced it with **AES-256-GCM** authenticated encryption, giving both confidentiality and a 128-bit integrity tag that makes ciphertext tampering detectable. The remaining six enhancements replace invented numbers with physics: Thorp acoustic absorption for delay, a 15 % Bernoulli loss channel with retransmission, per-node battery depletion, random-waypoint mobility for AUVs, a z -score detector for delay-injection attacks, and an extended set of result plots benchmarked against five prior schemes.

Separately, we rewrote the Scyther specification. Our first model expressed the chain as one monolithic five-role protocol and failed several agreement claims; restructuring it into four independent Needham–Schroeder–Lowe hops, with message tags and the responder identity inside the second message, produced **32 of 32 claims verified with no attacks** under a Dolev–Yao adversary in 0.19s. The final system authenticates a hop in 0.4ms of computation at 48.8 μ J per cycle, sustains authentication across a lossy channel, and reroutes automatically when a buoy fails.

Keywords: underwater acoustic networks, mutual authentication, elliptic curve cryptography, ECDH, AES-GCM, Scyther, BAN logic, Thorp absorption, replay resistance.

Acknowledgement

We thank **Prof. Mahendra Shukla**, Department of Information and Communication Technology, ABV-IIITM Gwalior, for supervising this work. His insistence that we justify every number in the simulation — rather than accept a plausible-looking constant — is directly responsible for the channel model in Chapter 6 being physical instead of arbitrary.

We are grateful to the authors of the base paper for a clearly specified protocol, and to Prof. Cas Cremers and the Scyther project for making rigorous protocol verification accessible to undergraduates. Finally, we thank the Institute for access to the IEEE Xplore digital library, without which the base paper would have been out of reach.

Contents

Abstract	1
Acknowledgement	3
List of Figures	7
List of Tables	8
1 Introduction	1
1.1 The setting	1
1.2 Problem statement	2
1.3 The base paper	3
1.4 Objectives of this project	3
1.5 What this report contributes beyond the paper	4
1.6 Organisation of the report	4
2 Preliminaries	6
2.1 One-way hash functions	6
2.2 Symmetric versus asymmetric encryption	6
2.3 Elliptic curve cryptography	7
2.3.1 The paper’s pentatope extension (PEECC)	7
2.4 ECDH: agreeing on a secret in public	8
2.5 Nonces, timestamps and the replay attack	9
2.6 Mutual authentication	9
2.7 The Dolev–Yao adversary	10
2.8 Notation used throughout	11
3 The Base Protocol	12
3.1 Network model	12
3.2 Overview of the three phases	13
3.3 Phase 1: System initialisation	13
3.4 Phase 2: Registration	14
3.4.1 Storage cost of the registration set	14
3.5 Phase 3: Mutual authentication	15
3.5.1 Steps in words	16

3.6	A worked example: one message end to end	17
3.6.1	Where the 2112 bits come from	18
3.7	Resilience through fallback paths	18
3.8	Long-term key management	19
4	Security Analysis and Performance in the Base Paper	21
4.1	Threat model	21
4.2	Formal analysis with BAN logic	21
4.2.1	Notation	21
4.2.2	Inference rules	22
4.2.3	Goals, assumptions and the derivation	22
4.3	Informal security propositions	23
4.4	Machine verification with Scyther	24
4.5	Performance in the base paper	25
4.5.1	Experimental setup	25
4.5.2	Communication cost	26
4.5.3	Computational cost	26
4.5.4	Energy per authentication cycle	26
5	Our Implementation	28
5.1	Deliverables	28
5.2	Architecture of the simulator	29
5.3	Network instantiation	29
5.4	The three phases in code	29
5.4.1	Phase 1 — initialisation	30
5.4.2	Phase 2 — registration	30
5.4.3	Phase 3 — mutual authentication	30
5.4.4	Routing with fallback	31
5.5	What was wrong with our first version	32
6	Enhancements	33
6.1	E1 — Replacing XOR with AES-256-GCM	33
6.1.1	Why the XOR cipher had to go	33
6.1.2	What AES-GCM provides	35
6.1.3	The code change	36
6.2	E2 — The Thorp acoustic absorption model	37
6.3	E3 — Packet loss with retransmission	38
6.4	E4 — Per-node battery depletion	39
6.5	E5 — AUV mobility	40

6.6	E6 — Anomaly detection on the delay stream	40
6.7	E7 — Extended metrics and comparison charts	41
7	Formal Verification with Scyther	42
7.1	What Scyther does	42
7.2	Attempt 1: the monolithic model, and why it failed	42
7.3	Attempt 2: four independent NSL hops	43
7.3.1	The Needham–Schroeder–Lowe pattern	44
7.3.2	Message tags	44
7.3.3	One hop in full	44
7.4	Verification results	45
7.5	Reading the result correctly	47
7.6	Attacks addressed, and by what	47
7.7	Reproducing the verification	47
8	Results	49
8.1	Console output	49
8.2	Network topology and the recovered route	50
8.3	Authentication delay	51
8.4	Energy consumption	52
8.5	Communication cost	53
8.6	Comparison against prior schemes	54
8.7	Throughput	55
8.8	Battery depletion	56
8.9	Summary of measured results	57
9	Discussion, Limitations and Conclusion	58
9.1	Where our implementation differs from the base paper	58
9.2	Limitations	60
9.3	Future work	60
9.4	Conclusion	61
	References	63

List of Figures

2.1	ECDH key agreement	8
2.2	Replay defeated by the freshness window	9
2.3	The Dolev–Yao adversary model	10
3.1	UWC network model with fallback routing	13
3.2	The three protocol phases	13
3.3	Complete twelve-message authentication chain	16
3.4	One message traced through every stage	18
3.5	Dynamic fallback rerouting	19
4.1	BAN logic derivation for hop 1	23
5.1	Simulator architecture	29
6.1	XOR versus AES-GCM	35
6.2	AES-GCM wire format	36
6.3	Retransmission on a lossy hop	39
6.4	Z-score delay anomaly detection	41
7.1	Restructuring the Scyther model	43
7.2	Scyther GUI verification output	46
8.1	Generated network topology	50
8.2	Delay versus number of nodes	51
8.3	Energy versus number of nodes	52
8.4	Communication cost versus number of nodes	53
8.5	Comparison with five prior schemes	54
8.6	Authentication throughput	55
8.7	Battery level per node	56

List of Tables

1.1	Terrestrial versus underwater acoustic channels	2
1.2	Division of work between the base paper and this project.	4
2.1	Symbols and entities, following the base paper’s Table II.	11
4.1	BAN logic notation used in the base paper’s proof.	22
4.2	The five BAN rules applied in the proof.	22
4.3	The paper’s five security propositions and the mechanism behind each.	24
4.4	Scyther claim types and what each rules out.	25
4.5	Communication cost comparison	26
4.6	Computational cost comparison	26
4.7	Energy per authentication cycle against prior schemes.	27
5.1	Repository contents.	28
5.2	Defects in our first implementation.	32
6.1	Summary of the seven enhancements.	33
6.2	Property-by-property comparison of the two ciphers.	37
6.3	Per-hop distances and resulting propagation delays.	38
7.1	Verification run.	45
7.2	Scyther claim results	46
7.3	Attack coverage across the whole system.	47
8.1	Consolidated results.	57
9.1	Deliberate divergences from the base paper.	59

Introduction

1.1 The setting

Underwater wireless communication networks (UWCNs) support environmental monitoring, oceanographic research, offshore oil and gas exploration, pipeline inspection and naval surveillance. A typical deployment scatters sensor nodes across the seabed, uses autonomous underwater vehicles (AUVs) or submarines as mobile collectors, floats buoys at the surface, and hands the data to a satellite that relays it to a base station ashore.

Everything unusual about security in this setting follows from one physical fact: **radio waves do not propagate through seawater**. Salt water is electrically conductive, and a conductor absorbs electromagnetic energy; a 2.4 GHz signal loses essentially all of its strength within a few centimetres. Underwater networks therefore fall back on the only carrier that travels usefully far in water — **sound**.

That single substitution invalidates every engineering assumption a network protocol normally makes. Table 1.1 contrasts the two regimes.

Table 1.1: The terrestrial radio channel compared with the underwater acoustic channel, and what each difference costs a security protocol.

Property	Terrestrial radio	Underwater acoustic	Consequence for a security protocol
Carrier	Electromagnetic	Pressure (sound)	—
Speed	3×10^8 m/s	≈ 1500 m/s	A 1 km hop costs 0.67 s one way; any handshake needing several round trips is unusable.
Bandwidth	100+ Mbps	≈ 10 kbps	Every bit is expensive. A 3000-bit handshake needs 300 ms just to clock out of the modem.
Latency	Milliseconds	Seconds	The replay-freshness window must be seconds wide, not milliseconds.
Packet loss	$< 1\%$	10–30 %	Retransmission must be designed in; a protocol that assumes delivery will stall.
Power	Mains or large battery	Small sealed cell, unserviceable	RSA and bilinear pairings drain a node in weeks.
Topology	Largely fixed	Drifting sensors, moving AUVs, failing buoys	The protocol must reroute without tearing down the session.

1.2 Problem statement

Conventional authentication — TLS, IPsec, Kerberos as ordinarily deployed — is not merely inefficient here, it is infeasible. A TLS 1.3 handshake exchanges several kilobytes over two round trips. At 10 kbps a 4 kB certificate chain alone occupies 3.2 s of channel time, before adding 0.67 s of flight per direction, and any lost fragment restarts the exchange. Multiply that across four hops and a single session consumes tens of seconds and a measurable fraction of the node’s lifetime energy budget.

RSA is equally unattractive: an RSA-2048 public key is 256 bytes where an elliptic-curve key of comparable strength is 32 bytes, and on this channel an eightfold larger key is an eightfold larger transmission bill. Conversely, the lightweight schemes proposed for underwater use often buy their efficiency by weakening security — omitting mutual authentication, skipping formal verification, or failing entirely when a relay node dies.

The gap. What is needed is a protocol that is simultaneously *lightweight* enough for a battery-powered acoustic node, *mutually authenticating* across a multi-hop relay chain, *resilient* to the loss of an intermediate node, and *formally verified* rather than merely argued to be secure.

1.3 The base paper

This report studies the protocol proposed by C. Rupa, G. S. Varshitha, D. Divya, T. R. Gadekallu and G. Srivastava, “A Novel and Robust Authentication Protocol for Secure Underwater Communication Systems”, *IEEE Internet of Things Journal*, vol. 12, no. 22, pp. 47519–47531, November 2025 [1].

The paper’s contributions, as stated by its authors, are:

1. A **pentatope-based elliptic curve cryptosystem** (PEECC) that reduces the number of global parameters from six to five and raises the difficulty of the discrete logarithm problem.
2. A **mutual authentication protocol** enforcing hop-wise freshness with nonces and timestamps, so replay is defeated even when propagation delay is large.
3. **Dynamic fallback authentication peers**, allowing communication to continue through a backup buoy or satellite during a partial outage.
4. **Formal validation** using BAN logic and the Scyther model checker, reporting no successful attack across more than sixty test cases.
5. A measured **30–34 % reduction in communication overhead** (2112 bits against 3008–3216 bits for prior schemes) and sub-millisecond authentication latency.
6. A **hierarchical key management and revocation** framework with small storage cost on constrained nodes.

1.4 Objectives of this project

1. Understand and set out the base protocol completely, including the cryptographic background needed to follow it, so that the report is self-contained for a reader who has not studied protocol security.
2. Implement the protocol as a working simulator over a realistic eight-node underwater topology, including the fallback mechanism.
3. Specify the protocol formally in SPDL and verify it with Scyther against a Dolev–Yao adversary.
4. Identify weaknesses in our own first implementation and correct them — in

particular, replace the placeholder cipher with real authenticated encryption and replace invented channel numbers with a physical propagation model.

5. Measure delay, energy, communication cost, throughput and battery depletion, and compare against the five prior schemes tabulated in the base paper.
6. State honestly where our implementation diverges from the paper.

1.5 What this report contributes beyond the paper

The base paper specifies a protocol and reports analytical costs. It does not publish an implementation. Our contribution is a reproducible one, summarised in Table 1.2.

Table 1.2: Division of work between the base paper and this project.

From the base paper	Added by this project
Protocol design: three phases, twelve messages, hop-wise freshness	Executable eight-node simulator (<code>uwc_simulation.py</code> , 406 lines)
Fallback peer concept	Working fallback: buoy B1 is forced to fail and traffic reroutes through B2 mid-run
Claim of Scyther verification (figure only)	Complete published SPDL model, 4 protocols, 32 claims, reproducible in 0.19 s
AES-128 named as the symmetric primitive	Actual AES-256-GCM authenticated encryption, replacing our own XOR placeholder
Delay derived from distance and sound speed	Thorp frequency-dependent absorption model plus multipath jitter
Loss-free analytical model	15 % Bernoulli per-hop loss with three-attempt retransmission
Static energy figure of 48.8 μJ per cycle	Per-node battery depletion tracked across rounds, reported as % remaining
Mobility discussed qualitatively	Random-waypoint drift on AUV nodes, feeding back into link delay
Delay injection listed as a threat	z -score anomaly detector on the observed delay stream
Two comparison tables	Seven generated plots including throughput, a metric absent from the paper

1.6 Organisation of the report

Chapter 2 builds the cryptographic background from scratch: hash functions, symmetric and asymmetric encryption, elliptic curves, ECDH, nonces, timestamps and the Dolev–

Yao adversary. Chapter 3 presents the base paper’s network model and its three phases, with a full message-sequence chart and a worked numerical example. Chapter 4 covers the paper’s security analysis — BAN logic, the informal propositions, Scyther — and its performance tables. Chapter 5 describes our implementation and, candidly, its initial flaws. Chapter 6 presents the seven enhancements, with Section 6.1 devoted to the cipher replacement. Chapter 7 describes the formal verification and the model restructuring that made it succeed. Chapter 8 reports the measured results. Chapter 9 states our deviations from the paper, the limitations, and the future work.

Preliminaries

This chapter assumes no background beyond ordinary programming. Every cryptographic idea the protocol relies on is introduced here with a concrete example, so that Chapters 3 onwards read without interruption.

2.1 One-way hash functions

A cryptographic hash function maps an input of *any* length to a fixed-length fingerprint:

$$H : \{0, 1\}^* \longrightarrow \{0, 1\}^n \quad (2.1)$$

SHA-256 produces $n = 256$ bits, conventionally written as 64 hexadecimal characters.

SHA256("underwater")	->	8e4c9d6f2b1a7053c8e4...	(64 hex characters)
SHA256("underwatex")	->	41f7a02c9db35e18770c...	(no visible relation)

Listing 2.1: Avalanche: a one-character change alters roughly half the output bits.

Three properties matter for this protocol:

- **Pre-image resistance (one-wayness).** Given the output, recovering the input is infeasible; exhaustive search costs about 2^{256} attempts.
- **Collision resistance.** Two distinct inputs with the same output cannot be found in practice.
- **Avalanche.** Flipping one input bit flips about half the output bits, so the digest carries no partial information about the input.

The protocol uses hashing for two distinct jobs: turning a public key into a short unforgeable *identity*, and condensing a raw ECDH shared point into a usable *symmetric key*.

2.2 Symmetric versus asymmetric encryption

Symmetric encryption uses the same key to encrypt and decrypt. AES is the standard. It is fast and cheap in both time and energy — exactly what a battery-powered sensor needs. Its difficulty is key distribution: both parties must already share the key, over a channel an adversary is listening to.

Asymmetric (public-key) encryption uses a key *pair*: a public key published freely, and a private key never revealed. Anything encrypted under the public key can be opened only with the matching private key. This solves distribution but costs two to three orders of magnitude more computation.

Practical systems, including this one, are **hybrid**: asymmetric mathematics establishes a shared secret, and fast symmetric encryption then protects the actual messages.

2.3 Elliptic curve cryptography

An elliptic curve over a finite field $\mathbb{F}_p$ is the set of points (x, y) satisfying

$$y^2 \equiv x^3 + ax + b \pmod{p} \quad (2.2)$$

together with a point at infinity. Our implementation uses **secp256r1** (NIST P-256), for which $p \approx 2^{256}$.

Curve points can be added by a geometric rule, and adding a base point G to itself k times is written $k \cdot G$ — **scalar multiplication**. This single operation is the entire engine of ECC, because it is easy in one direction and infeasible in the other:

- Computing $k \cdot G$ from k takes about 256 double-and-add steps — microseconds.
- Recovering k from $k \cdot G$ is the **Elliptic Curve Discrete Logarithm Problem** (ECDLP). The best known attack needs roughly 2^{128} operations on a 256-bit curve.

A **private key** is therefore just a random integer $k \in [1, n - 1]$, and the matching **public key** is the curve point $k \cdot G$. Publishing that point discloses nothing about k .

Why ECC and not RSA here. ECC reaches 128-bit security with a 256-bit key; RSA needs 3072 bits for the same. On a 10 kbps acoustic link, transmitting a 3072-bit key costs 307 ms against 26 ms for ECC — and that gap is paid on every hop, every session, out of a battery that cannot be replaced.

2.3.1 The paper's pentatope extension (PEECC)

The base paper proposes an extension it calls *pentatope elliptic curve cryptography*. Instead of a two-dimensional point group, it defines a five-dimensional group $G_5 = \{(x_1, \dots, x_5) \mid x_i \in \mathbb{F}_p\}$ over the curve $\psi = X^3 + Y^3 + Z^3 + W^3 + T^3 = dXYZWT$. Key generation follows standard ECC but folds in the *pentatope number*, the fifth

entry of a Pascal's-triangle diagonal,

$$S = \frac{S(S+1)(S+2)(S+3)}{24}, \quad K = S^2 + S + 41, \quad (2.3)$$

which the authors use to derive a key value internally rather than importing an externally computed shared secret. Their claim is that the discrete logarithm problem in G_5 has solving complexity around $2^{5n/2}$ against $2^{2n/2}$ for a conventional curve, at a measured $1.3\times$ scalar-multiplication overhead on an ARM Cortex-A53, and that the number of global parameters falls from six to five.

Scope note. Our implementation uses standard secp256r1, *not* PEECC. The pentatope construction is not accompanied in the paper by a reference implementation or concrete curve parameters, so reproducing it was outside what we could verify. Everything we measure is therefore a lower bound on the paper's claimed security, not a reproduction of it. This is discussed in Section 9.1.

2.4 ECDH: agreeing on a secret in public

Elliptic Curve Diffie–Hellman lets two nodes derive an identical secret without ever transmitting it. It rests on one algebraic fact — scalar multiplication commutes:

$$a \cdot (b \cdot G) = ab \cdot G = b \cdot (a \cdot G) \quad (2.4)$$

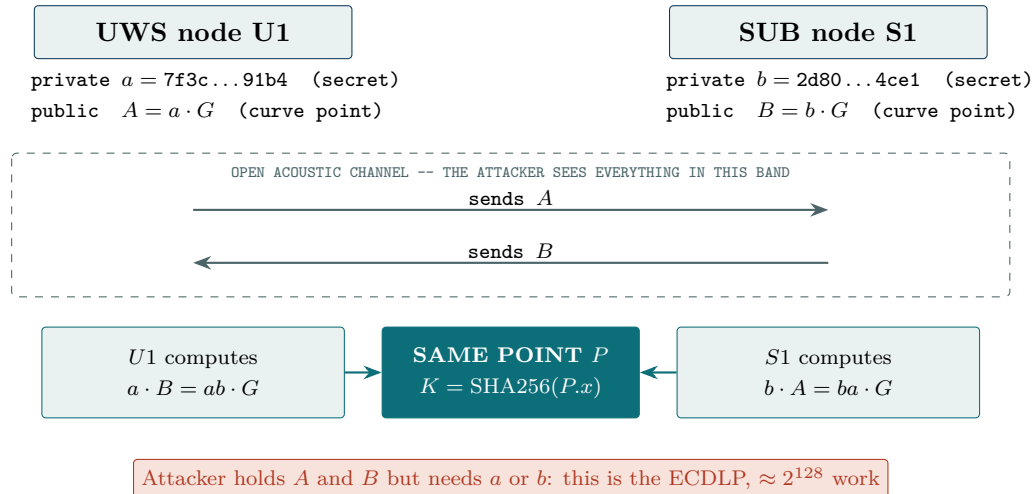


Figure 2.1: ECDH key agreement. Only public points cross the channel. Each node multiplies the peer's point by its own private scalar and arrives at the identical curve point P ; hashing P 's x -coordinate yields a 256-bit symmetric key. The key itself is never transmitted, so an eavesdropper who records the entire exchange still has to solve the discrete logarithm problem.

In our simulator this is six lines of Python:

```

1 def gen_keys():
2     pk = random.randint(1, curve.field.n - 1)    # private scalar
3     return pk, pk * curve.g                      # public point
4
5 def shared(priv, pub):
6     pt = priv * pub                             # ECDH: a.B on one side, b.A on
7     the other
8     return hashlib.sha256(str(pt.x).encode()).hexdigest()

```

Listing 2.2: Key generation and ECDH shared-secret derivation (uwc_simulation.py, lines 109–118).

2.5 Nonces, timestamps and the replay attack

A **nonce** is a “number used once”: a fresh random value included in a message so that no two runs of the protocol ever look alike. A **timestamp** records when the message was created. Together they defeat the **replay attack**, which requires no cryptography at all — the adversary simply records a legitimate ciphertext and re-sends it later. She cannot read it, but if the receiver accepts it she has impersonated a legitimate node.

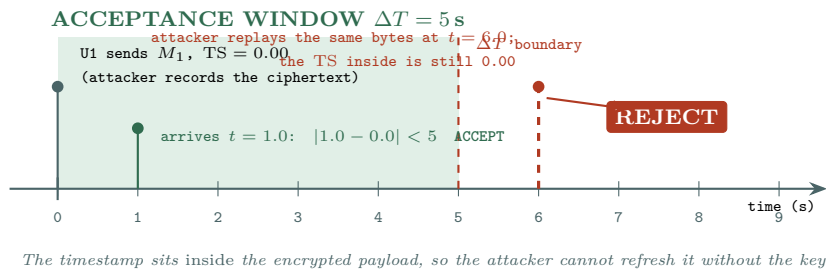


Figure 2.2: Why replay fails. The receiver compares its own clock against the timestamp carried inside the ciphertext. Because the adversary cannot decrypt, she cannot update that timestamp; the stale value falls outside ΔT and the message is dropped. The nonce is the second line of defence, catching a replay that arrives while still inside the window. ΔT is measured in seconds here, not milliseconds, precisely because acoustic propagation is slow.

2.6 Mutual authentication

One-way authentication means the server proves its identity — that is what a browser padlock provides. **Mutual** authentication means both sides prove identity to each other. In a sensor network both directions matter: a buoy must not accept readings from a counterfeit sensor, and a sensor must not surrender its data to a counterfeit buoy.

The proof always has the same shape: *I send you a fresh challenge that only the real you could answer, and you return it inside something only you could have produced.*

2.7 The Dolev–Yao adversary

Formal analysis assumes the strongest realistic adversary, named after Dolev and Yao [6]. She is not an attacker sitting on one link — she *is* the network. She may read every message, delete messages, reorder them, replay old ones, and inject anything she can construct from what she already knows. What she cannot do is break cryptography: without the key she cannot decrypt, and without a private key she cannot forge. This is the *perfect cryptography* assumption.

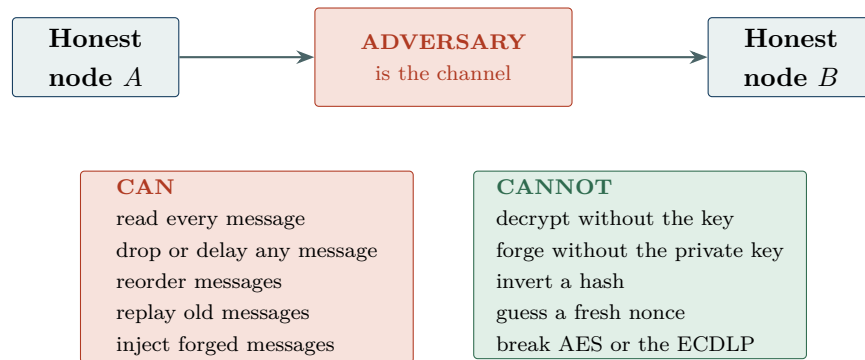


Figure 2.3: The Dolev–Yao adversary. She controls the entire network but not the mathematics. A protocol that survives this model has no flaw in its *message design*; any remaining weakness lies in the primitives or the implementation. This is exactly the model Scyther checks against in Chapter 7.

2.8 Notation used throughout

Table 2.1: Symbols and entities, following the base paper’s Table II.

Symbol	Meaning
UWS	Underwater sensor: measures temperature, pressure, salinity, current velocity
SUB	Submarine or AUV: mobile relay aggregating data from several sensors
B	Surface buoy: bridges water to air, carries a GPS module
SAT	Satellite: long-haul link from buoy to shore
BS	Base station on land: final destination and trust anchor
PK_i	Private key of entity i (a secret scalar)
PU_i	Public key of entity i , $PU_i = PK_i \times \rho$ (a curve point)
ρ	Generator point of the curve
ID_i	Identity of i , derived as a hash of PU_i
RID_i	Registration identifier, $H(ID_i \parallel PU_i \parallel PK_i)$
ϑ_i	Verification value, $H(RID_i \parallel PU_i)$
$Nonce_i$	Fresh random number generated by i for one run
TS_n	Timestamp attached to message n
ΔT	Freshness window; accept only if $ \text{now} - TS < \Delta T$
$E\{PU_j, \langle m \rangle\}$	m encrypted so only the holder of PK_j can read it
$D\{PK_j, (\cdot)\}$	Decryption with the private key of j
$\parallel$	Concatenation of fields
R	Set of registered RIDs held by a node (hash table or Bloom filter)
$S_{i \leftrightarrow j}$	Session established between i and j
ENV	Aggregated environmental payload {Temp, Press, CurrVel, Sal}

The Base Protocol

This chapter sets out the protocol exactly as proposed by Rupa *et al.* [1]: the network it assumes, its three phases, all twelve messages, and a worked example following one real message from the seabed to the shore.

3.1 Network model

The network is a five-layer hierarchy. Data flows upward from the seabed to land, and every adjacent pair authenticates each other before any payload moves.

- **UWS** — underwater sensors on the seabed, measuring temperature (T), salinity (SAL), pressure (P) and current velocity (V).
- **SUB** — a submarine or AUV acting as a mobile hub that aggregates readings from several sensors.
- **B** — surface buoys relaying between the submarine and the satellite. Buoys carry GPS, so they contribute location as well as environmental data.
- **SAT** — satellites providing the long-distance link to shore.
- **BS** — the base station on land, which processes the data and acts as the trust anchor for revocation.

Crucially, the model builds in **redundancy**: each submarine keeps links to more than one buoy, and each buoy can reach an alternate satellite. This is what makes the fallback mechanism of Section 3.7 possible.

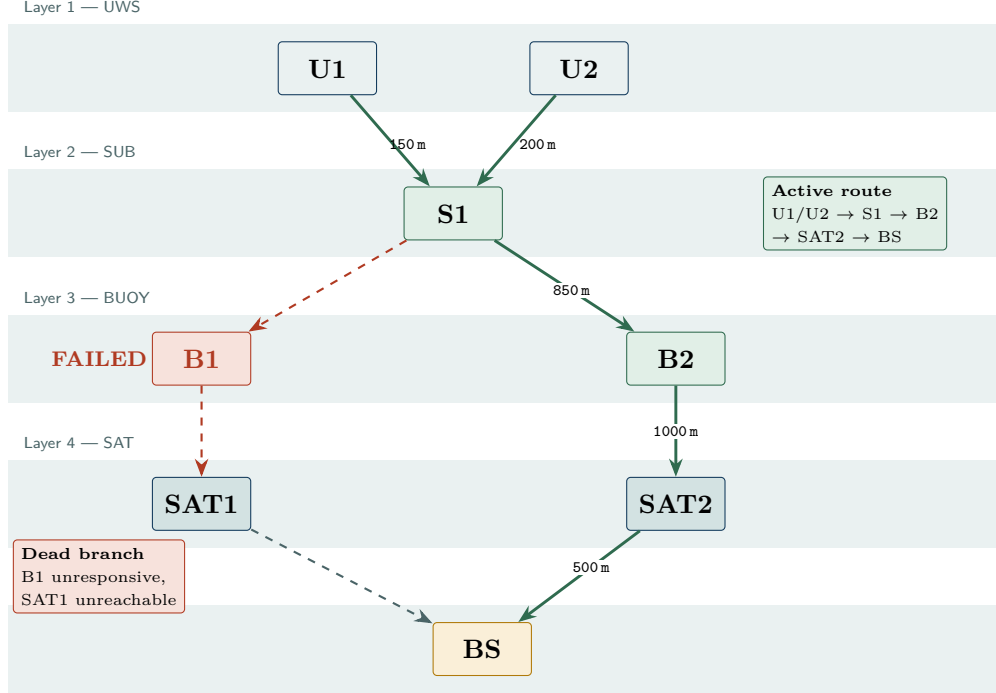


Figure 3.1: The network model as we instantiate it. Eight nodes over five layers. Buoy B1 is forced to fail in the simulation (dashed red), so the submarine reroutes through the pre-registered fallback buoy B2 and its satellite SAT2 (solid green). Distances are the inter-node values used in our channel model (Section 6.2).

3.2 Overview of the three phases

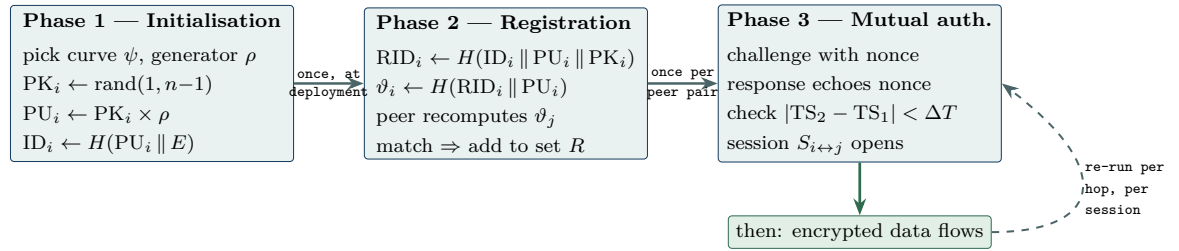


Figure 3.2: The three phases. Initialisation and registration happen once; mutual authentication runs on every hop of every session and is where the protocol’s per-message cost lives.

3.3 Phase 1: System initialisation

Every entity $E \in \{\text{UWS}, \text{SUB}, \text{B}, \text{SAT}, \text{BS}\}$ performs six steps:

1. Select the pentatope elliptic curve $\psi = X^3 + Y^3 + Z^3 + W^3 + T^3 = dXYZWT$ over $\mathbb{F}_p$ for a large prime p and constant d .
2. Choose a generator point $\rho = (X_\rho, Y_\rho, Z_\rho, W_\rho, T_\rho)$ on ψ of large prime order n .

3. Generate a private key $PK_i \in \langle 1, n-1 \rangle$ uniformly at random.
4. Compute the public key $PU_i = PK_i \times \rho$, where $\times$ denotes scalar multiplication on ψ .
5. Derive the identifier by hashing the public key, $ID_i = H(PU_i \parallel E)$. Because H is one-way and collision-resistant, an identity cannot be forged without the corresponding key pair.
6. Publish $M_i = (ID_i, PU_i)$ to the entities it must talk to.

Why derive the identity from the key. If identities were assigned arbitrarily, an attacker could claim any identity she liked. Binding $ID_i = H(PU_i \parallel E)$ means that to claim an identity you must possess a public key that hashes to it — and to *use* that identity you must hold the matching private key. Impersonation therefore reduces to inverting a hash or solving the ECDLP. This is the mechanism behind Proposition 1 in Section 4.3.

3.4 Phase 2: Registration

Registration binds an entity to the network and gives its peers a way to detect a tampered or duplicate registration.

1. E computes its registration identifier and a verification value:

$$RID_i = H(ID_i \parallel PU_i \parallel PK_i), \quad \vartheta_i = H(RID_i \parallel PU_i) \quad (3.1)$$

2. E sends the registration message $M_i = \{RID_i, PU_i, \vartheta_i\}$ to the peer j .
3. j extracts $\{RID_i, PU_i\}$ and independently recomputes $\vartheta_j = H(RID_i \parallel PU_i)$. If $\vartheta_j \neq \vartheta_i$ the message has been altered in transit and is rejected as INVALID.
4. If the message is valid but $RID_i \in R$, the registration is a duplicate and is refused.
5. Otherwise RID_i is added to R and a session $S_{i \leftrightarrow j}$ is created.

Note that RID_i mixes in the *private* key PK_i , so no other party can compute another node's RID ; but because H is one-way, publishing RID_i does not leak PK_i .

3.4.1 Storage cost of the registration set

The paper is careful about memory on constrained nodes. The set R is a hash table or Bloom filter giving $O(1)$ lookup, and each entity stores only the subset relevant to its own neighbourhood — a sensor keeps the identifiers of its submarines, not of the whole network. With 160-bit RID values and k direct peers, storage is about $20k$ bytes; for a sensor with fewer than ten peers this is under 250 bytes. Centralised nodes

such as the BS hold the full $O(N)$ record while constrained nodes hold an $O(k)$ partial view with $k \ll N$.

3.5 Phase 3: Mutual authentication

This is the operational core. The chain from seabed to shore is completed in twelve messages, in a pattern that repeats identically at each of the four hops:

- a **challenge** carrying the initiator's identity, a fresh nonce and a timestamp;
- an **acknowledgement** carrying the responder's identity, the *same* nonce echoed back, and a new timestamp;
- a **data message** carrying the payload once the session is open.

Four hops $\times$ three messages = twelve. Figure 3.3 is the complete sequence.

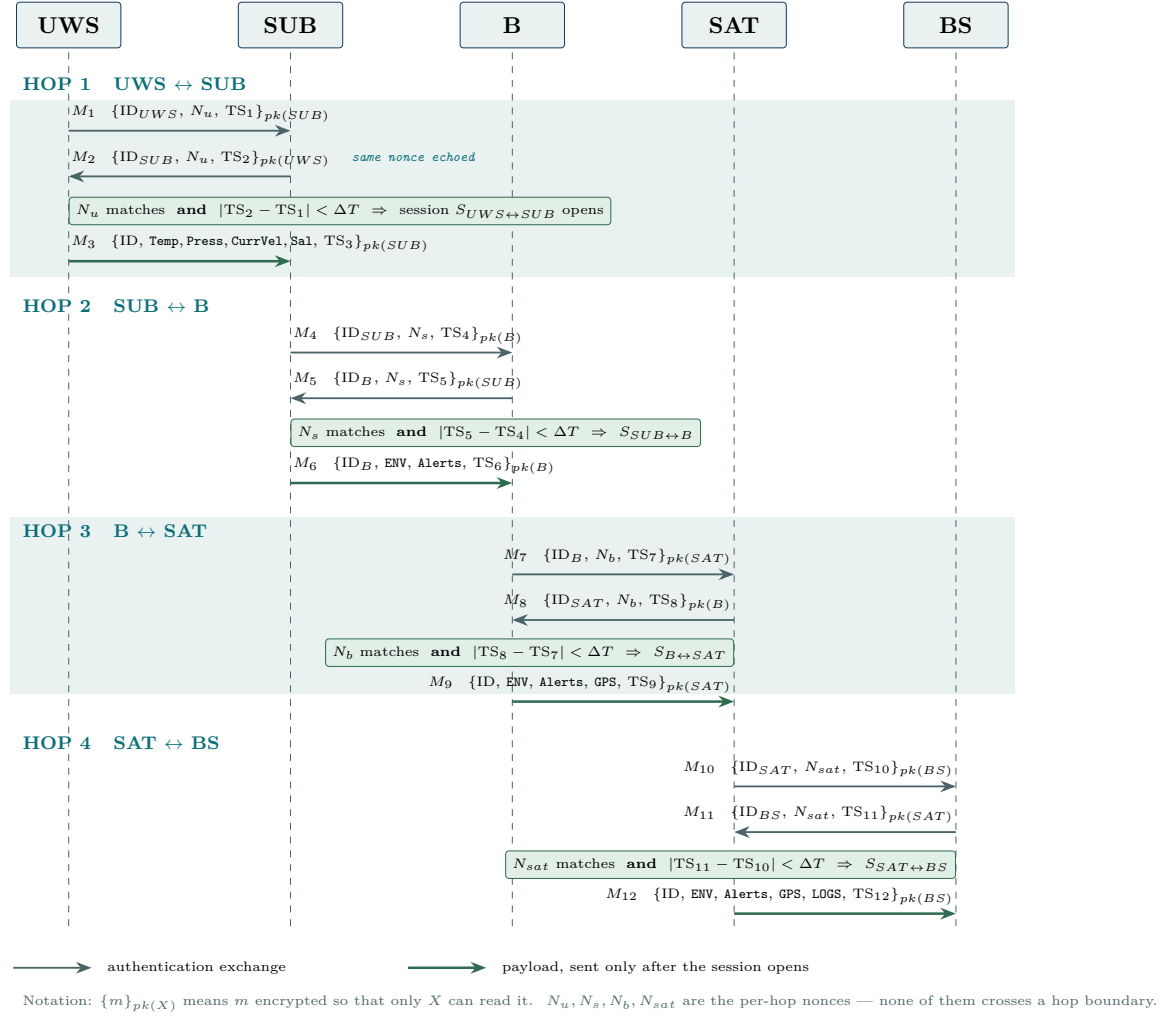


Figure 3.3: The full protocol, M_1 through M_{12} . Each hop repeats the same three-message pattern: challenge, acknowledgement echoing the nonce, then payload. Sessions are established *independently* per hop — the sensor never talks to the base station directly, and no nonce crosses a hop boundary. The payload grows as it climbs: sensor readings at hop 1, plus alerts at hop 2, plus GPS at hop 3, plus logistics at hop 4.

3.5.1 Steps in words

- M1.** UWS $\rightarrow$ SUB. The sensor encrypts $\langle ID_{UWS} \parallel \text{Nonce}_{UWS} \parallel TS_1 \rangle$ under the submarine's public key. Only the submarine, holding PK_{SUB} , can decrypt — so a successful decryption is already evidence the message was intended for it.
- M2.** SUB $\rightarrow$ UWS. The submarine replies with its own identity, *the same nonce*, and a fresh timestamp, encrypted under the sensor's public key. Echoing the nonce is the proof of liveness: only a party that could decrypt M_1 knows what to echo.
- M3.** Session check. UWS verifies $\text{Nonce}_{SUB}^{recv} = \text{Nonce}_{UWS}^{sent}$ and $|TS_2 - TS_1| < \Delta T$. Both must hold. The session $S_{UWS \leftrightarrow SUB}$ opens, and UWS sends the sensor payload $\langle \text{Temp}, \text{Press}, \text{CurrVel}, \text{Sal} \rangle$ as M_3 . SUB aggregates it into ENV.

- M4–M6.** The identical pattern runs between SUB and B, with SUB’s nonce. At M_6 the submarine forwards ENV plus *Alerts* — notifications of jamming, spoofing or eavesdropping detected lower down.
- M7–M9.** The pattern runs between B and SAT. At M_9 the buoy adds its *GPS* coordinates to the payload.
- M10–M12.** The pattern runs between SAT and BS. At M_{12} the satellite adds machine and logistics updates (*LOGS*), and the base station receives the complete record.

The central design decision. Freshness is verified *per hop*. Each hop generates its own nonce and validates its own timestamp difference. Nothing propagates end-to-end, and there is no global session state to track. This is what makes the protocol viable underwater: a lost packet costs one hop’s retry, not a whole-chain restart, and the 1.7s end-to-end propagation delay never enters a freshness check.

3.6 A worked example: one message end to end

Abstract notation hides how little actually happens per message. Figure 3.4 traces M_1 from sensor U1 to submarine S1 using the real field sizes and real value shapes our simulator produces.

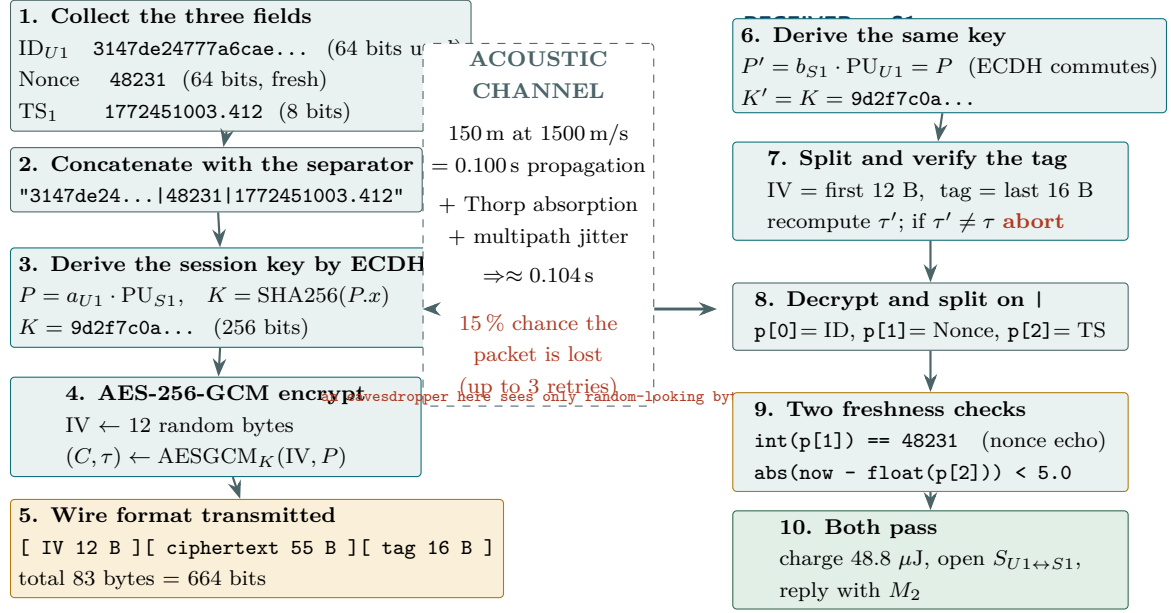


Figure 3.4: M_1 from U_1 to S_1 , stage by stage. The left column is what the sender does, the right column what the receiver does, and the centre is the physical channel. Steps 3 and 6 produce the same key without it ever crossing the channel. Steps 4 and 7 are our AES-GCM substitution (Section 6.1); the base paper writes this stage as a public-key encryption under PU_{SUB} , which is cryptographically equivalent in effect and cheaper in practice. Note that the tag is checked at step 7 *before* any plaintext is trusted.

3.6.1 Where the 2112 bits come from

The paper's headline communication figure is 2112 bits for a complete authentication cycle. Its accounting uses standardised field sizes: 64-bit identities, 64-bit nonces, 8-bit timestamps, 160-bit hash outputs, and 512-bit representations for ECC points or AES keys. Summing the fields transmitted across the login and authentication phases of all four hops gives 2112 bits, against 3008–3216 bits for the five comparison schemes. At a nominal modem rate of 10 kbps, 2112 bits occupy

$$t_{tx} = \frac{2112 \text{ bits}}{10\,000 \text{ bits/s}} = 211 \text{ ms}, \quad (3.2)$$

which is the reference line drawn on our delay plot in Chapter 8.

3.7 Resilience through fallback paths

A relay in the sea is not a reliable component. Buoys are struck by shipping, drift out of position, flood, or simply run down. The base paper's answer is *dynamic reassignment to fallback authentication peers*.

If a buoy B_1 stops responding, the submarine reinitiates mutual authentication

with an alternate buoy B_2 taken from its neighbour list. Likewise a buoy can route to a secondary satellite when the primary fails or is delayed. The essential detail is that these fallback entities are **pre-registered during system initialisation** and held in a trusted cache. Rerouting therefore does not require a new registration exchange — the public keys and registration identifiers are already known — and the same nonce-freshness and timestamp bounds validate the rerouted session as would validate an original one.

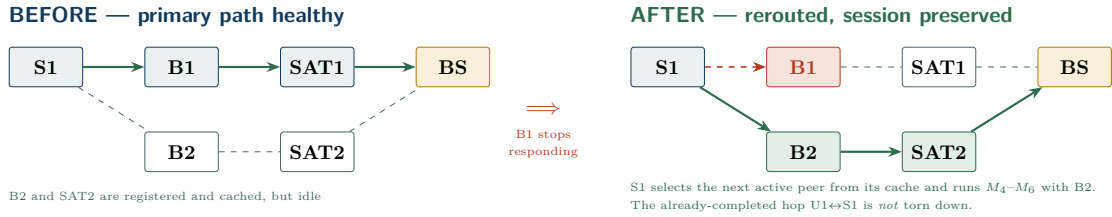


Figure 3.5: Fallback in action. Because B_2 was registered at initialisation, the submarine can authenticate with it immediately; only the three messages of the failed hop are repeated. Nothing upstream is affected, which is the direct benefit of authenticating per hop rather than end to end.

The paper reports that a new session can be established in under 1 ms thanks to the lightweight cryptographic operations and pre-distributed public keys, so a node that drifts into a new submarine’s range can simply re-run M_1 – M_3 rather than waiting on a recovery procedure. Ongoing sessions remain valid as long as latency stays within ΔT and loss stays below threshold.

3.8 Long-term key management

Three provisions in the paper address cryptographic sustainability over a deployment lifetime.

1. **Secure private-key storage.** Private keys are held in tamper-resistant hardware — a secure element such as the Microchip ATECC608A, or a TPM — where cost and power allow. Where they do not, the paper falls back on firmware-level obfuscation and memory isolation to raise the cost of physical extraction.
2. **Key revocation.** The base station periodically issues a signed revocation list naming compromised or expired nodes. It is broadcast to the network, and every entity checks incoming messages against it before initiating or answering an authentication. The list itself is authenticated with a management key held by all trusted nodes.
3. **Initial public-key distribution.** At deployment each node uploads its public key to the base station over a trusted link — a physical interface, or a short-range

line-of-sight acoustic channel. The key is bound to the identity by

$$\text{ID}_i = H(\text{PU}_i \parallel E \parallel \text{DeviceSerial}) \quad (3.3)$$

which prevents identity spoofing, and the keys are then disseminated to the other entities so mutual authentication can proceed.

Security Analysis and Performance in the Base Paper

Having set out what the protocol *does*, this chapter covers the paper’s argument that it is *secure*, and its accounting of what it costs. Three separate arguments are made: a formal proof in BAN logic, a set of informal propositions, and machine verification with Scyther.

4.1 Threat model

The paper assumes an adversary capable of:

- **Passive eavesdropping** — recording everything on the channel.
- **Active impersonation** — claiming to be a legitimate node.
- **Replay** — retransmitting captured messages.
- **Delay injection** — deliberately slowing packets to push a legitimate exchange outside its timestamp window, causing desynchronisation.
- **Physical tampering** — compromising a subset of nodes and extracting their stored credentials.

The four defences claimed against these are: unique non-reusable nonces with adaptive timestamp validation; asymmetric encryption with secure key derivation; tamper-resistant storage with hierarchical credential management; and revocation combined with dynamic fallback paths.

4.2 Formal analysis with BAN logic

BAN logic [9] is a formal system for reasoning about *belief* in authentication protocols. Rather than modelling bits, it models what each participant is entitled to conclude after seeing a message. It is a proof technique, not a tool: you write down what each party initially believes, apply inference rules to each message, and check whether the desired beliefs follow.

4.2.1 Notation

For entities ε, ϕ and messages M_ε, M_ϕ :

Table 4.1: BAN logic notation used in the base paper's proof.

Notation	Reading
$\prec M_\varepsilon \succ$	Message M_ε is <i>fresh</i>
$\varepsilon \Leftarrow M_\varepsilon$	ε <i>sees</i> the statement
$\varepsilon \Rightarrow M_\varepsilon$	ε has <i>jurisdiction</i> over the statement
$\varepsilon \gg M_\varepsilon$	ε <i>believes</i> the statement
$\varepsilon \sim M_\varepsilon$	ε <i>once said</i> the statement
$\{M_\varepsilon\}_{\text{PU}_\varepsilon}$	Encrypted under ε 's public key
$\phi \xleftrightarrow{\text{PU}_\varepsilon} \varepsilon$	The two communicate securely using PU_ε

4.2.2 Inference rules

Table 4.2: The five BAN rules applied in the proof.

	Rule	Plain reading
R1	Message-meaning	If I believe we share a key and I see a message encrypted under it, then I believe you sent that message.
R2	Nonce-verification	If I believe the message is fresh and that you sent it, then I believe you currently believe it (not that you said it long ago).
R3	Jurisdiction	If I believe you are authoritative over a statement and that you believe it, then I believe it.
R4	Freshness	If any part of a message is fresh, the whole message is fresh.
R5	Belief	If I believe you believe a compound message, I believe you believe each part.

4.2.3 Goals, assumptions and the derivation

Eight goals are stated, one per direction of each of the four hops:

$$\begin{aligned}
\Omega_1 : \text{UWS} \gg (\text{UWS} \xleftrightarrow{\text{PU}_{\text{SUB}}} \text{SUB}) & \quad \Omega_2 : \text{SUB} \gg (\text{UWS} \xleftrightarrow{\text{PU}_{\text{UWS}}} \text{SUB}) \\
\Omega_3 : \text{SUB} \gg (\text{SUB} \xleftrightarrow{\text{PU}_B} \text{B}) & \quad \Omega_4 : \text{B} \gg (\text{SUB} \xleftrightarrow{\text{PU}_{\text{SUB}}} \text{B}) \\
\Omega_5 : \text{B} \gg (\text{B} \xleftrightarrow{\text{PU}_{\text{SAT}}} \text{SAT}) & \quad \Omega_6 : \text{SAT} \gg (\text{B} \xleftrightarrow{\text{PU}_B} \text{SAT}) \\
\Omega_7 : \text{SAT} \gg (\text{SAT} \xleftrightarrow{\text{PU}_{\text{BS}}} \text{BS}) & \quad \Omega_8 : \text{BS} \gg (\text{SAT} \xleftrightarrow{\text{PU}_{\text{SAT}}} \text{BS})
\end{aligned}$$

The initial assumptions are that every entity believes its own nonces and timestamps

are fresh (A_1, A_2), that each entity has jurisdiction over its own identity (A_3), and that public keys are correctly distributed (A_4).

The derivation then runs for thirty steps. Figure 4.1 shows the first hop, which is representative — the other three hops repeat the identical five-step shape.

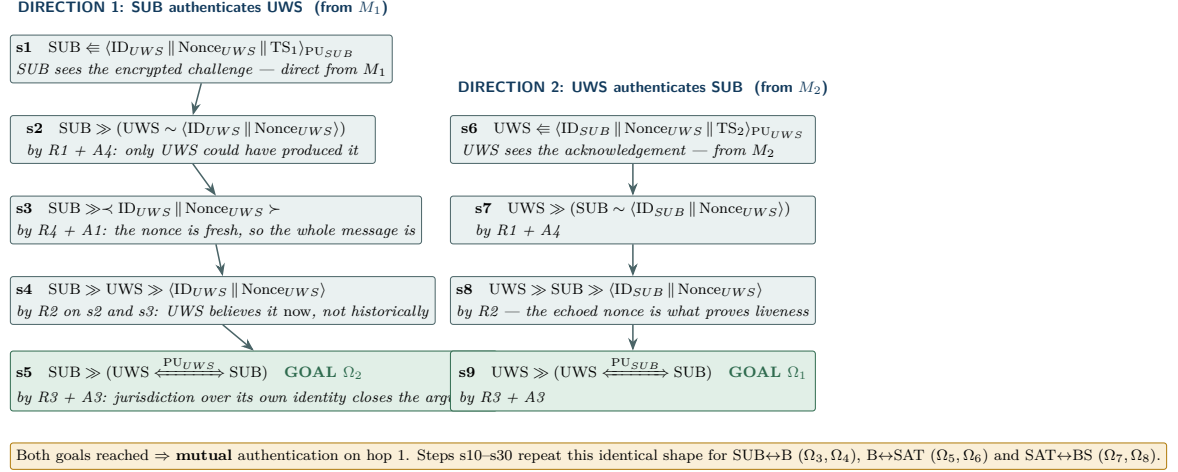


Figure 4.1: The BAN derivation, hop 1. The left column is the challenge direction, the right the acknowledgement direction. Reaching both goals is what makes the authentication *mutual*. The critical step is s4/s8, where the freshness of the nonce upgrades “ ϕ once said this” into “ ϕ believes this now” — without freshness, a replayed message would satisfy the earlier steps.

4.3 Informal security propositions

The paper supplements the BAN proof with five propositions.

Table 4.3: The paper’s five security propositions and the mechanism behind each.

	Claim	Mechanism
P1	No identity spoofing	Each identity is bound to a key pair and validated by signature at registration; $\text{Verify}(\text{Sig}(\text{ID}_i, \text{SK}_i), \text{PK}_i)$ must return true. Forging an identity means forging a signature.
P2	Replay resistance	The receiver checks $N_i \notin \text{UsedNonces}$ and $ \text{now} - \text{TS}_i \leq \Delta T$. The paper further proposes an <i>adaptive</i> ΔT : each node keeps a rolling average of recent inter-node delays and adjusts the window, so legitimate variation is tolerated while stale injection is still caught.
P3	No man-in-the-middle	Messages are encrypted under the recipient’s public key. Only the holder of the matching private key can decrypt or meaningfully alter them.
P4	Secure key establishment	$K_{ij} = \text{DH}(\text{PK}_i, \text{SK}_j) = \text{DH}(\text{PK}_j, \text{SK}_i)$. Only the two legitimate parties can compute it. Bounded ΔT per hop additionally causes a delay-injection attempt to abort the session.
P5	Resists stolen-verifier and ephemeral-secret-leakage attacks	Long-term private keys authenticate session establishment only; session keys are ephemeral, derived from fresh nonces and timestamps, and never stored. Compromising a node therefore yields neither past nor future session keys — this is forward secrecy in effect. The registration set is periodically checked against the BS’s revocation list.

4.4 Machine verification with Scyther

Scyther [10] is an automated model checker for security protocols developed by Cas Cremers. You describe the protocol in SPDL (Security Protocol Description Language), state the properties you want, and the tool searches exhaustively for a trace in which a Dolev–Yao adversary violates one. If no such trace exists within the bound, it reports a proof of correctness; if one does, it draws the attack.

The four properties checked are:

Table 4.4: Scyther claim types and what each rules out.

Claim	Guarantee	Attack it rules out
Secret	The named value is never learned by the adversary: $\forall \mathcal{A}, \Pr[\mathcal{A}(S) = S] \leq \theta$	Eavesdropping, key recovery
Alive	The claimed peer actually executed the protocol	Talking to a ghost
Weakagree	If ε_A believes it ran with ε_B , then ε_B really did run with ε_A : $P(\varepsilon_A, \varepsilon_B) \Rightarrow \exists P(\varepsilon_B, \varepsilon_A)$	Man-in-the-middle, impersonation
Niagree / Nisynch	Both parties agree on all exchanged values, in a causally ordered sequence: a message received at t_r was sent at $t_s \leq t_r$	Replay, message reordering, value substitution

The paper reports that all entities passed, configured for five roles with up to five concurrent sessions each, across more than sixty test cases, with no attacks found. Its Fig. 6 shows the tool output. The paper does not publish its SPDL source; our own reconstruction, and the restructuring it needed, is the subject of Chapter 7.

4.5 Performance in the base paper

4.5.1 Experimental setup

The paper’s evaluation uses a custom Python 3.10 simulator on Ubuntu 22.04, Intel Core i7-11800H, 32 GB RAM. The modelled network is a hierarchical tree of 10 to 200 nodes over 2 to 5 depth levels, with acoustic links at a nominal 10 kbps, inter-node distances of 100 to 1000 m and sound speed 1500 m/s. Message fields are standardised at 64-bit identities, 64-bit nonces, 8-bit timestamps, 160-bit hashes and 512-bit ECC-point or AES-key representations.

4.5.2 Communication cost

Table 4.5: Communication cost per full authentication cycle, in bits (base paper, Table IV). “Ref [21]–[25]” are the scheme identifiers used in the base paper’s own comparison, and are *not* the reference numbers of this report.

Protocol	Entity 1	Entity 2	Entity 3	Entity 4	Total	vs proposed
Ref [21]	896	768	512	832	3008	+42 %
Ref [22]	1024	832	512	832	3200	+52 %
Ref [23]	960	800	544	832	3136	+49 %
Ref [24]	912	784	512	832	3040	+44 %
Ref [25]	1008	848	528	832	3216	+52 %
Proposed	640	512	320	640	2112	baseline

The percentages are relative to the proposed scheme as denominator; for example $(3008 - 2112)/2112 = 42\%$.

4.5.3 Computational cost

Table 4.6: Computational cost per entity in milliseconds (base paper, Table V). T_{exp} exponentiation, T_{ecm} elliptic-curve multiplication, T_h hash, T_{pair} bilinear pairing.

Protocol	UWS / BS	SUB / SAT	Operations
Ref [21]	0.536	0.800	2 exp + 6 hash
Ref [22]	19.70	25.000	3 ECM + 4 ECM + pairings
Ref [23]	75.88	90.000	3 ECM + 2 pairings
Ref [24]	2.352	2.900	1 ECM + 4 hash
Ref [25]	1.245	1.600	1 ECM + 3 hash
Proposed	0.400	0.500	1 ECC + 2 hash + 4 AES

The saving comes from eliminating bilinear pairings and multiple exponentiations — the two operations that dominate cost in the older schemes — and replacing them with one optimised ECC operation plus cheap symmetric work. Against Ref [23] this is a $189\times$ improvement.

4.5.4 Energy per authentication cycle

The paper’s energy accounting uses published ARM Cortex-M4 / Cortex-A53 benchmarks: one ECC operation $\approx 24\mu\text{J}$, one hash $\approx 6\mu\text{J}$, one AES-128 encryption or

decryption $\approx 3.2 \mu\text{J}$. A single entity performs one ECC operation, two hashes, and two encryptions plus two decryptions:

$$E_{\text{cycle}} = \underbrace{24}_{\text{ECC}} + \underbrace{2 \times 6}_{\text{hash}} + \underbrace{(2 + 2) \times 3.2}_{\text{AES}} = 24 + 12 + 12.8 = 48.8 \mu\text{J} \quad (4.1)$$

Table 4.7: Energy per authentication cycle against prior schemes.

Protocol	Energy	Relative
Ref [22]	$\approx 2300 \mu\text{J}$	$47\times$ more
Ref [23]	$\approx 8900 \mu\text{J}$	$182\times$ more
Ref [24]	$\approx 280 \mu\text{J}$	$5.7\times$ more
Proposed	$48.8 \mu\text{J}$	baseline

At $48.8 \mu\text{J}$ — under 0.05 mJ — a node with a 1000 mAh cell at 3.3 V ($3.3 \text{ J} = 3.3 \times 10^6 \mu\text{J}$) could in principle perform about $67\,000$ authentication cycles on cryptography alone. Chapter 8 shows what happens once transmission and reception energy are added, which is where the real budget goes.

Our Implementation

The base paper specifies a protocol but publishes no code. This chapter describes what we built, how it is organised, and — candidly — what was wrong with our first version. The corrections are the subject of Chapter 6.

5.1 Deliverables

Table 5.1: Repository contents.

File	Purpose
<code>uwc_simulation.py</code>	The simulator: 406 lines. ECC key generation, registration, four-hop authentication with fallback, AES-GCM, the Thorp channel, loss and retransmission, battery tracking, mobility, anomaly detection, and all seven plots.
<code>uwc_protocol.spdl</code>	Scyther model: four protocols, eight roles, 32 claims.
<code>requirements.txt</code>	<code>tinyec</code> , <code>cryptography</code> , <code>networkx</code> , <code>matplotlib</code> , <code>numpy</code> , <code>scipy</code> .
<code>scyther/</code>	The bundled Scyther 1.3.0 distribution, so verification is reproducible without a separate download.
<code>launch_scyther_gui.bat</code>	One-click launcher for the bundled GUI.
<code>output/</code>	The seven generated plots plus the GUI verification screenshot.

5.2 Architecture of the simulator

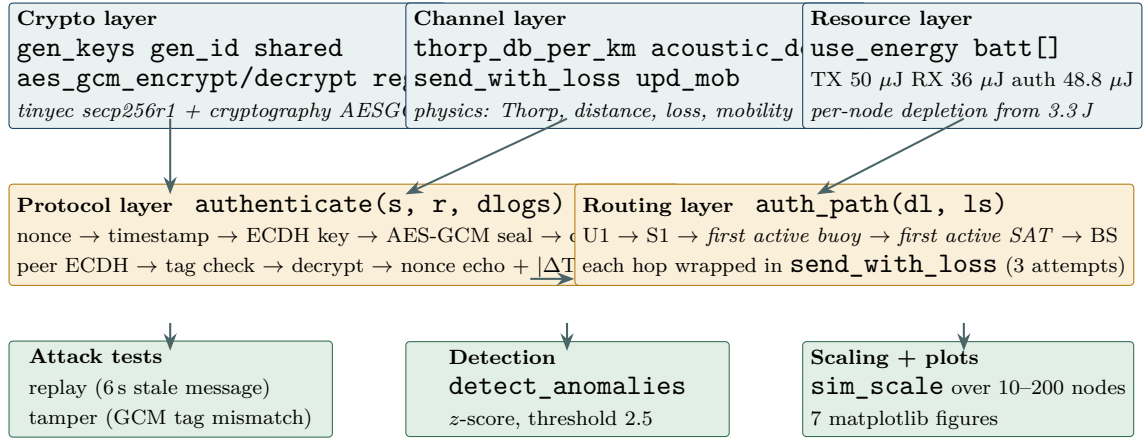


Figure 5.1: How `uwc_simulation.py` is organised. Three support layers feed the protocol layer, which the routing layer drives hop by hop. The analysis row consumes the resulting delay stream, energy ledger and success record.

5.3 Network instantiation

Eight nodes over five layers, matching Figure 3.1:

```

1 nodes = {
2     "UWS": ["U1", "U2"],
3     "SUB": ["S1"],
4     "BUOY": ["B1", "B2"],
5     "SAT": ["SAT1", "SAT2"],
6     "BS": ["BS"],
7 }
8 edges = [
9     ("U1", "S1"), ("U2", "S1"), ("S1", "B1"), ("S1", "B2"),
10    ("B1", "SAT1"), ("B2", "SAT2"), ("SAT1", "BS"), ("SAT2", "BS"),
11 ]
12
13 def is_node_active(node):
14     return node != "B1" # B1 forced-failed to demonstrate fallback

```

Listing 5.1: Topology definition (lines 30–43).

Buoy B1 is deliberately marked dead. This is not a bug: it is how we exercise the fallback mechanism on every run, so that the reported route is always the recovered one.

5.4 The three phases in code

5.4.1 Phase 1 — initialisation

Each node gets a random private scalar, the corresponding public point, and an identity derived from that point:

```

1 def gen_keys():
2     pk = random.randint(1, curve.field.n - 1)
3     return pk, pk * curve.g
4
5 def gen_id(pub):
6     return hashlib.sha256((str(pub.x) + str(pub.y)).encode()).hexdigest()
7
8 for g in nodes:
9     for n in nodes[g]:
10        pk, pub = gen_keys()
11        node_data[n] = {"private": pk, "public": pub, "id": gen_id(pub)}
```

Listing 5.2: Initialisation (lines 109–128).

5.4.2 Phase 2 — registration

```

1 def reg(n):
2     d = node_data[n]
3     return hashlib.sha256(
4         (d["id"] + str(d["public"]) + str(d["private"])).encode()).hexdigest()
5
6 regs = {n: reg(n) for n in node_data}
```

Listing 5.3: Registration identifier (lines 148–152).

This is the paper’s $RID_i = H(ID_i \parallel PU_i \parallel PK_i)$ directly.

5.4.3 Phase 3 — mutual authentication

```

1 TSW = 5.0    # timestamp freshness window, seconds
2
3 def authenticate(s, r, dlogs=None):
4     if batt.get(s, 1) <= 0 or batt.get(r, 1) <= 0:
5         return False                                # dead node cannot
6                                                         participate
7     nc = random.randint(10000, 99999)                # fresh nonce
8     ts = time.time()                                # timestamp
9     sk = shared(node_data[s]["private"], node_data[r]["public"]) # ECDH, sender
10                                                         side
11     ct = aes_gcm_encrypt(f"{node_data[s]['id']}|{nc}|{ts}", sk) # seal
12     pd = acoustic_delay(s, r, mob_off=mob_off.get(s, 0)) # channel delay
13     if dlogs is not None:
14         dlogs.append(pd)
15     use_energy(s, "tx"); use_energy(r, "rx")
16     rk = shared(node_data[r]["private"], node_data[s]["public"]) # ECDH,
17                                                         receiver side
18     try:
19         dec = aes_gcm_decrypt(ct, rk)
```

```

17     except Exception:
18         print(f" {r}: Decryption FAILED (tampered message)"      # GCM tag
              rejected it
19         return False
20     p = dec.split("|")
21     if abs(time.time() - float(p[2])) < TSW and int(p[1]) == nc:  # freshness +
        nonce echo
22         use_energy(r, "auth")
23         return True
24     return False

```

Listing 5.4: The authentication routine (lines 175–197). This is the heart of the simulator.

Two checks gate acceptance, and both must pass: the timestamp must be within $\Delta T = 5$ s, and the nonce must match what was sent. The `try/except` around decryption is where GCM’s integrity tag does its work — a tampered ciphertext raises before any plaintext is produced.

5.4.4 Routing with fallback

```

1 def auth_path(dl, ls):
2     upd_mob()
3     af = lambda s, r: authenticate(s, r, dl)
4
5     if not send_with_loss(af, "U1", "S1", ls):
6         return False
7     b = next((x for x in nodes["BUOY"] if is_node_active(x)), None)  # B1 dead
        -> B2
8     if not b:
9         return False
10    print(f" Using BUOY: {b}")
11    if not send_with_loss(af, "S1", b, ls):
12        return False
13    sv = next((x for x in nodes["SAT"] if is_node_active(x)), None)
14    if not sv:
15        return False
16    print(f" Using SAT: {sv}")
17    if not send_with_loss(af, b, sv, ls):
18        return False
19    return send_with_loss(af, sv, "BS", ls)

```

Listing 5.5: Path selection with automatic fallback (lines 199–217).

The `next(...)` generator expression is the fallback: it walks the buoy list and takes the first active one. Because B1 is dead, every run selects B2 and the route becomes $U1 \rightarrow S1 \rightarrow B2 \rightarrow SAT2 \rightarrow BS$ without any restart or renegotiation. Sessions on the hops already completed are untouched.

5.5 What was wrong with our first version

Our initial implementation (commit 82d9b09 and earlier) ran end to end and produced plausible graphs. It was also, in five specific ways, not a faithful or safe model. We list them here because Chapter 6 exists to fix them.

Table 5.2: Defects in our first implementation.

	Defect	Why it mattered
D1	Repeating-key XOR used as the cipher	Cryptographically broken. Two messages under the same key leak $P_1 \oplus P_2$; a single known plaintext recovers the key outright. It also provides no integrity at all, so any tampering goes undetected. See Section 6.1.
D2	<code>delay = random.uniform(0.04, 0.08)</code>	The delay had no physical basis. It did not depend on distance, on frequency, or on the medium — so the delay plot conveyed nothing about underwater propagation.
D3	100 % packet delivery assumed	The single most characteristic property of the underwater channel — 10–30 % loss — was absent, so the protocol was never tested under the condition it was designed for.
D4	Energy computed but never charged	<code>energy_consumption()</code> returned the constant 48.8 and nothing consumed it. There was no notion of a node running down.
D5	All nodes static; no detection; three basic plots	AUVs that never move, no defence against delay injection, and no comparison against the prior schemes the paper tabulates.

The XOR problem, stated plainly. The line we shipped first was:

```
encrypt = lambda m,k: "".join(chr(ord(c)^ord(k[i%len(k)])) for i,c in
                                enumerate(m))
```

This is a Vigenère cipher over bytes. It looks like encryption — the output is unreadable — but it is defeated by a first-year exercise, and it silently accepts any modification an attacker makes to the ciphertext. Presenting it as the realisation of the paper’s “AES-128 symmetric encryption” would have been wrong. Section 6.1 is the correction.

Enhancements

Seven changes take the simulator from a demonstration to something defensible. The first is a security fix; the next four replace invented numbers with physics or accounting; the last two add capability the base paper only describes.

Table 6.1: Summary of the seven enhancements.

	Enhancement	Replaces	Why
E1	AES-256-GCM authenticated encryption	Repeating-key XOR	Confidentiality <i>and</i> integrity; matches the paper’s stated use of AES
E2	Thorp absorption model	<code>random.uniform(0.04, 0.08)</code>	Delay becomes a function of distance and frequency
E3	15 % Bernoulli loss with 3 retries	Assumed perfect delivery	Tests the protocol under the channel’s defining property
E4	Per-node battery depletion	Static energy constant	Shows which nodes wear out and how fast
E5	Random-waypoint AUV mobility	Static nodes	Link distance, hence delay, varies per round as it would in the sea
E6	z -score anomaly detection	Nothing	Detects delay-injection attacks the paper names but does not counter
E7	Comparison charts and new metrics	3 basic plots	Benchmarks against the five prior schemes; adds throughput

6.1 E1 — Replacing XOR with AES-256-GCM

This is the most important change in the project, so it gets the most space.

6.1.1 Why the XOR cipher had to go

A repeating-key XOR cipher encrypts by exclusive-or-ing each plaintext byte with a byte of the key, cycling the key when it runs out. Decryption is the same operation, because $x \oplus k \oplus k = x$. It has two fatal properties.

Attack 1: known plaintext recovers the key immediately. The attacker knows the message format — "`<id>|<nonce>|<timestamp>`" — and the identity field is a

public SHA-256 hex string she can read off the network during registration. Given any plaintext byte p_i and its ciphertext byte c_i :

$$k_{i \bmod |k|} = p_i \oplus c_i \quad (6.1)$$

Recovering the whole key needs only as many known bytes as the key is long. She then decrypts every past and future message on that link.

Attack 2: key reuse leaks the relationship between messages. The simulator derives one ECDH key per node pair and reuses it for every message on that link. For two messages under the same key:

$$C_1 \oplus C_2 = (P_1 \oplus K) \oplus (P_2 \oplus K) = P_1 \oplus P_2 \quad (6.2)$$

The key vanishes. Because the two plaintexts share a rigid format — the same 64-character identity prefix, the same separators, timestamps differing only in their final digits — the XOR of the two plaintexts is highly structured, and standard crib-dragging recovers both.

And there is no integrity at all. XOR is malleable. Flipping bit j of the ciphertext flips bit j of the plaintext. The attacker cannot read the message, but she can *change* it in a controlled way, and the receiver has no means of noticing. She can alter the nonce, or move the timestamp, without knowing the key.

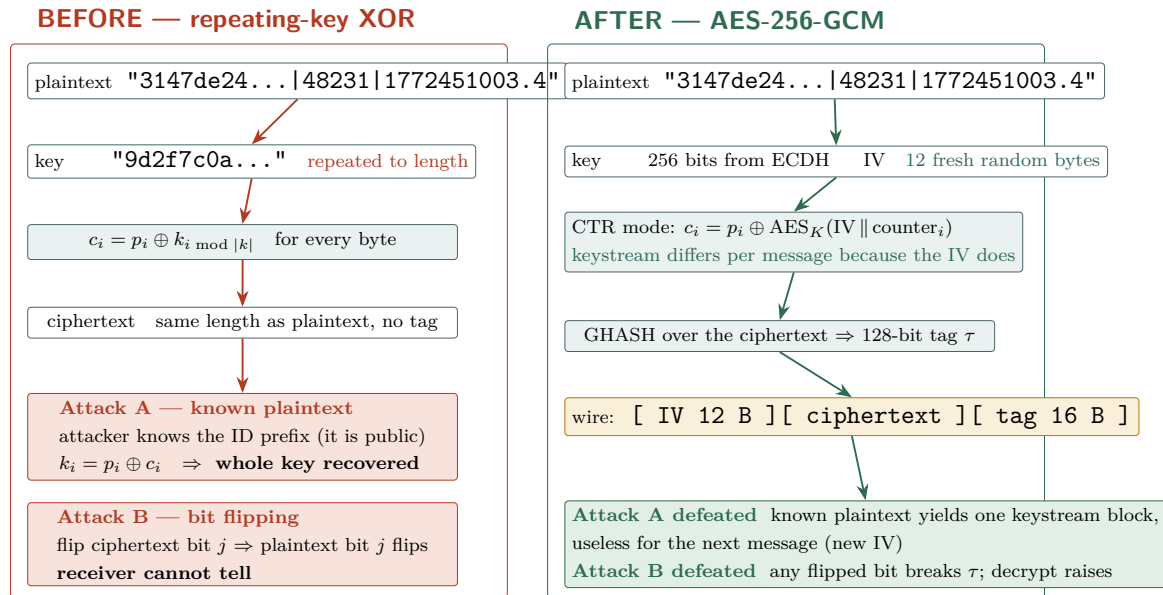


Figure 6.1: The cipher replacement, and what each attack does to each scheme. The structural difference is on the right: a fresh random IV per message means the keystream never repeats even though the ECDH key is long-lived, and the GHASH tag makes tampering detectable rather than silent. Both are absent from XOR by construction.

6.1.2 What AES-GCM provides

AES-GCM [11] is an *authenticated encryption with associated data* (AEAD) construction. It does two jobs in one pass:

- **Confidentiality** via AES in Counter (CTR) mode. AES-256 encrypts a counter block, and the resulting keystream is XOR-ed with the plaintext. Note that the final combining step *is* an XOR — the difference from our old code is that the keystream is the output of a block cipher and never repeats, rather than a short key cycled over and over.
- **Integrity** via GHASH, a polynomial hash over the ciphertext in $\text{GF}(2^{128})$, producing a 128-bit authentication tag. The receiver recomputes the tag and compares. Any modification — one bit — makes the tags differ, and decryption fails before any plaintext is released.

The 96-bit random IV is what breaks the key-reuse problem. The same long-lived ECDH key can protect any number of messages, because each message's keystream is derived from a different IV.

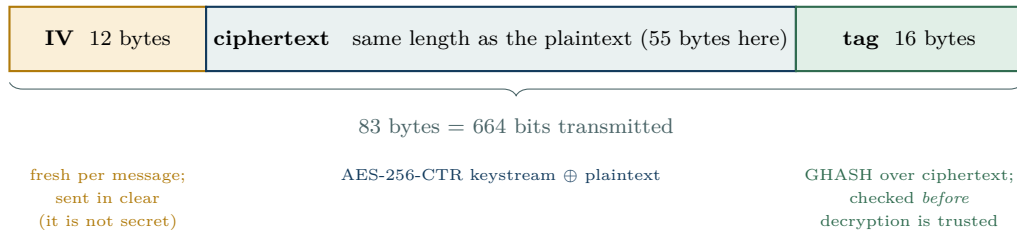


Figure 6.2: What actually goes on the wire. The IV is prepended so the receiver can reconstruct the keystream; it is public by design. The 28 bytes of overhead (IV plus tag) are the price of authenticated encryption, and at 10 kbps cost 22 ms per message — an acceptable trade for detecting tampering.

6.1.3 The code change

```
def encrypt(message, key):
    return ''.join(chr(ord(c) ^ ord(key[i % len(key)])) for i, c in enumerate(
        message))

def decrypt(cipher, key):
    return ''.join(chr(ord(c) ^ ord(key[i % len(key)])) for i, c in enumerate(
        cipher))
```

Listing 6.1: Before: the broken cipher.

```
1 from cryptography.hazmat.primitives.ciphers.aead import AESGCM
2
3 def aes_gcm_encrypt(plaintext, key_hex):
4     key = bytes.fromhex(key_hex[:64]) # 256-bit key from the ECDH digest
5     nonce = os.urandom(12) # 96-bit random IV, fresh per message
6     ct = AESGCM(key).encrypt(nonce, plaintext.encode(), None)
7     return nonce + ct # prepend the IV for the receiver
8
9 def aes_gcm_decrypt(ciphertext, key_hex):
10    key = bytes.fromhex(key_hex[:64])
11    return AESGCM(key).decrypt(ciphertext[:12], ciphertext[12:], None).decode()
```

Listing 6.2: After: AES-256-GCM (uwc_simulation.py, lines 50–58).

`AESGCM.decrypt` raises `InvalidTag` if the tag does not verify. The caller in `authenticate()` catches it and fails the hop:

```
try:
    dec = aes_gcm_decrypt(ct, rk)
except Exception:
    print(f" {r}: Decryption FAILED (tampered message)")
    return False
```

Listing 6.3: Tamper detection at the call site (lines 188–192).

Relationship to the base paper. The paper writes each message as $E\{\text{PU}_{\text{recv}}, \langle \cdot \rangle\}$ — encryption under the recipient’s public key — and separately states in its energy analysis that the symmetric primitive is AES-128, at $3.2 \mu\text{J}$ per operation. Our construction is the standard hybrid realisation of that: ECDH under the recipient’s public key derives the session key, and AES-256-GCM protects the payload. This is what public-key encryption schemes such as ECIES do internally. We use AES-256 rather than AES-128 because the ECDH digest is already 256 bits, and on this hardware the cost difference is negligible relative to the acoustic transmission.

Table 6.2: Property-by-property comparison of the two ciphers.

Property	Repeating-key XOR	AES-256-GCM
Confidentiality	Broken by known plaintext	256-bit security
Integrity	None	128-bit GHASH tag
Tamper detection	Impossible	<code>InvalidTag</code> on any change
Key reuse across messages	Catastrophic ($C_1 \oplus C_2 = P_1 \oplus P_2$)	Safe, given a fresh IV
Malleability	Fully malleable (bit flipping)	Non-malleable
Ciphertext expansion	0 bytes	28 bytes (12 IV + 16 tag)
Standardised	No	NIST SP 800-38D
Energy per operation	negligible	$\approx 3.2 \mu\text{J}$ (paper’s figure)

6.2 E2 — The Thorp acoustic absorption model

Sound in seawater does not merely spread out; it is *absorbed*, and the absorption depends strongly on frequency. Thorp’s empirical model [2] gives the absorption coefficient in decibels per kilometre for a frequency f in kHz:

$$\alpha(f) = \frac{0.11f^2}{1 + f^2} + \frac{44f^2}{4100 + f^2} + 2.75 \times 10^{-4}f^2 + 0.003 \quad (6.3)$$

The four terms correspond to boric acid relaxation, magnesium sulphate relaxation, pure water viscosity, and a low-frequency residual. Typical underwater modems operate between 10 and 25 kHz; our simulation uses 25 kHz.

The one-way delay of a hop is then propagation plus an absorption-derived jitter term plus a multipath spread term:

```

1 SOUND_MPS = 1500.0    # speed of sound in seawater
2
3 DISTS = {             # realistic inter-node distances, metres
4     ("U1","S1"):150,   ("U2","S1"):200,   ("S1","B1"):800,   ("S1","B2"):850,
5     ("B1","SAT1"):1000, ("B2","SAT2"):1000, ("SAT1","BS"):500, ("SAT2","BS"):500,
6 }
7
8 def thorp_db_per_km(f_khz):
9     f2 = f_khz ** 2
10    return 0.11*f2/(1+f2) + 44*f2/(4100+f2) + 2.75e-4*f2 + 0.003
11
12 def acoustic_delay(s, r, freq_khz=25.0, mob_off=0.0):
13     key = (s,r) if (s,r) in DISTS else (r,s)
14     dist = max(10, DISTS.get(key, 500) + abs(mob_off))    # mobility changes the
15                                                         # distance
16     prop = dist / SOUND_MPS                                # geometric propagation
17     jit = thorp_db_per_km(freq_khz) * (dist/1000) * 1e-4
18     mp = abs(random.gauss(0, 0.005))                      # multipath spread
19     return prop + jit + mp

```

Listing 6.4: Physical delay model (lines 66–85).

Table 6.3: Per-hop distances and resulting propagation delays.

Hop	Distance	Propagation delay
UWS → SUB	150–200 m	0.10–0.13 s
SUB → BUOY	800–850 m	0.53–0.57 s
BUOY → SAT	1000 m	0.67 s
SAT → BS	500 m	0.33 s
Full chain	2450–2550 m	≈1.63–1.70 s one way

Note the consequence: the end-to-end round trip is over three seconds. This is precisely why the protocol authenticates per hop — a ΔT of 5 s is comfortable for a single hop but would be marginal end to end.

6.3 E3 — Packet loss with retransmission

Stojanovic [3] measured 10–30 % packet loss in open-water acoustic channels. We model this as an independent Bernoulli trial per transmission at a conservative 15 %, with up to three attempts.

```

1 LOSS_RATE = 0.15
2 MAX_RETRY = 3
3
4 def send_with_loss(fn, s, r, ls):
5     for attempt in range(1, MAX_RETRY+1):
6         if random.random() < LOSS_RATE:
7             ls["lost"] += 1

```

```

8         print(f" [LOSS] {s}->{r} attempt {attempt}/{MAX_RETRY}")
9         continue
10        return fn(s, r)
11    ls["failed"] += 1
12    print(f" [FAIL] {s}->{r} link down after {MAX_RETRY} retransmissions")
13    return False

```

Listing 6.5: Lossy channel wrapper (lines 92–104).

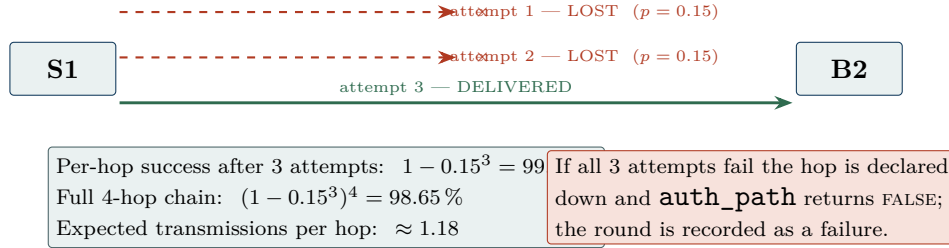


Figure 6.3: Three attempts per hop. With independent 15 % loss, three attempts raise per-hop reliability to 99.66 % and end-to-end reliability across all four hops to 98.65 %. This is what makes the protocol usable on a channel that drops one packet in seven.

6.4 E4 — Per-node battery depletion

Each node begins with a 1000 mAh cell at 3.3 V:

$$E_0 = 1000 \text{ mAh} \times 3.3 \text{ V} = 3.3 \text{ J} = 3\,300\,000 \mu\text{J} \quad (6.4)$$

Every operation now debits the ledger, rather than being counted in the abstract.

```

1 BAT_uJ = 3_300_000.0
2 AUTH_uJ, TX_uJ, RX_uJ = 48.8, 50.0, 36.0
3 batt = {n: BAT_uJ for g in nodes for n in nodes[g]}
4
5 def use_energy(node, op="auth"):
6     cost = {"auth": AUTH_uJ, "tx": TX_uJ, "rx": RX_uJ}.get(op, AUTH_uJ)
7     batt[node] -= cost
8     if batt[node] <= 0:
9         print(f" [DEAD] {node} battery depleted!")
10        return False
11    return True

```

Listing 6.6: Energy accounting (lines 135–145).

The 48.8 μJ authentication figure is Equation 4.1 from the base paper. The transmit and receive figures reflect acoustic modem power, and they dominate: a single hop costs 50 (TX) + 36 (RX) + 48.8 (auth) = 134.8 μJ , of which the cryptography is only 36 %. This is worth stating plainly, because it reframes the optimisation target — *reducing message size matters more than reducing cipher cost*, which is exactly why

the paper’s 2112-bit figure is its most valuable result.

6.5 E5 — AUV mobility

Submarines and AUVs move at roughly 2 m/s. Between authentication rounds we drift the mobile nodes (U1, U2, S1) by up to ± 10 m, clamped to ± 50 m from the nominal position, following the random-waypoint family surveyed by Camp *et al.* [14].

```

1 mob_off = {n: 0.0 for g in nodes for n in nodes[g]}
2
3 def upd_mob():
4     for n in ["U1", "U2", "S1"]:
5         mob_off[n] += random.uniform(-10, 10)
6         mob_off[n] = max(-50, min(50, mob_off[n]))

```

Listing 6.7: Mobility update (lines 163–168).

The offset feeds straight into `acoustic_delay`, so link delay varies from round to round exactly as it would at sea. This is the source of the non-monotonic shape in the delay plot of Chapter 8, and it is a feature, not noise.

6.6 E6 — Anomaly detection on the delay stream

The base paper names *delay injection* in its threat model: an adversary deliberately slows packets to push a legitimate exchange outside its ΔT window and force a desynchronisation. It proposes an adaptive ΔT as the countermeasure but does not implement a detector. We added one.

Every observed per-hop delay is recorded. A delay more than 2.5 standard deviations from the mean is flagged:

$$z_i = \frac{d_i - \mu}{\sigma}, \quad \text{flag if } |z_i| > 2.5 \quad (6.5)$$

```

1 def detect_anomalies(delays, thresh=2.5):
2     if len(delays) < 4:
3         return []
4     m = statistics.mean(delays)
5     s = statistics.stdev(delays)
6     return [] if s == 0 else [i for i, d in enumerate(delays) if abs((d-m)/s) >
                               thresh]

```

Listing 6.8: z -score detector (lines 224–229).

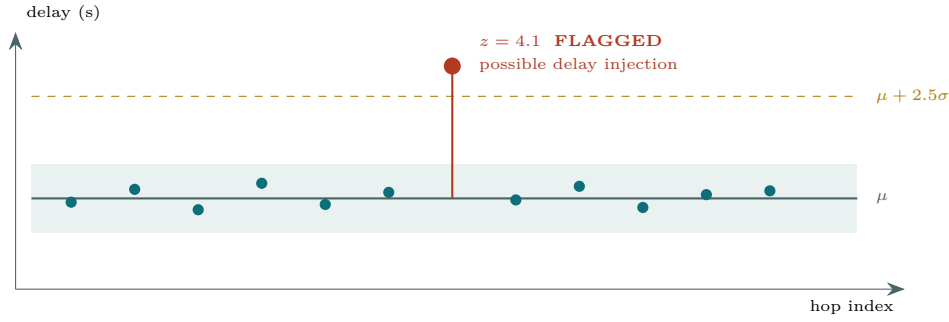


Figure 6.4: Detecting a delay-injection attempt. Ordinary variation from mobility and multipath stays within a couple of standard deviations. An artificially delayed packet stands out as a statistical outlier and is flagged, even though it may still be inside the ΔT window and would otherwise be accepted.

6.7 E7 — Extended metrics and comparison charts

The original produced three plots showing delay, energy and communication cost against node count, with no external reference. We extended this to seven, adding:

- comparison bar charts against all five prior schemes from the paper’s Tables IV and V, on a log scale for computational cost;
- reference lines on the energy and communication plots so the reader can see the margin rather than infer it;
- a network topology plot showing the failed node and the recovered route;
- **throughput** (authentications per second = $1/\bar{d}$), a metric absent from the base paper;
- **battery level per node** after the simulation, showing which nodes bear the cost.

All seven are presented and discussed in Chapter 8.

Formal Verification with Scyther

Arguing informally that a protocol is secure is unreliable; the Needham–Schroeder public-key protocol was believed correct for seventeen years before Lowe found an attack on it with a model checker [7]. This chapter describes how we specified our protocol in SPDL, why our first model failed, and what restructuring made all claims verify.

7.1 What Scyther does

Scyther [10] takes a protocol description and a set of claims, and searches exhaustively for a trace in which a Dolev–Yao adversary (Figure 2.3) violates a claim. Within its bound it either finds an attack and draws it, or returns a proof of correctness. It reasons symbolically about unbounded numbers of concurrent sessions, which is where subtle attacks live — an adversary running two sessions in parallel and relaying between them.

7.2 Attempt 1: the monolithic model, and why it failed

Our first specification mirrored the paper’s presentation literally: one protocol, `UWCFull`, with all five roles, symmetric keys $k(A,B)$, and nonces accumulating along the chain.

```
protocol UWCFull(UWS, SUB, BUOY, SAT, BS)
{
  role SUB {
    var Nu: Nonce; fresh Ns: Nonce; var Nb: Nonce;
    recv_1(UWS, SUB, {UWS, Nu}k(UWS,SUB));
    send_2(SUB, UWS, {SUB, UWS, Nu, Ns}k(UWS,SUB));
    send_3(SUB, BUOY, {SUB, UWS, Nu, Ns}k(SUB,BUOY));      /* Nu crosses the hop
    */
    recv_4(BUOY, SUB, {BUOY, SUB, UWS, Nu, Ns, Nb}k(SUB,BUOY));
    claim(SUB, Alive); claim(SUB, Niagree); claim(SUB, Secret, Ns);
  }
  /* ... BUOY, SAT, BS roles similarly accumulate every prior nonce ... */
}
```

Listing 7.1: First attempt (commit 82d9b09, since replaced). Nonces from earlier hops are carried forward into later ones.

Three problems followed from this shape.

1. **Nonces crossed hop boundaries.** N_u , generated by the sensor, appeared in

messages between SUB and BUOY. This contradicts the base paper’s own design — Section IV-C states explicitly that hop-wise freshness “eliminates the need for end-to-end nonce propagation” — and it widens the exposure of every nonce to every downstream node.

2. **Agreement claims failed.** With five roles interleaved in one protocol, **Niagree** could not be established: the adversary can run partial sessions and recombine message fragments across roles, because nothing in the message content ties a fragment to the hop it belongs to.
3. **Symmetric keys $k(A,B)$ misrepresented the protocol.** The paper uses public-key encryption under the recipient’s key, not a pre-shared symmetric key.

7.3 Attempt 2: four independent NSL hops

The rewrite applied one insight: *the base protocol is not one five-party protocol, it is four independent two-party protocols that happen to run in sequence*. Modelling it that way is both more faithful and easier to verify.

Each hop was written as a **Needham–Schroeder–Lowe** exchange — the classic three-message mutual authentication, with Lowe’s fix.

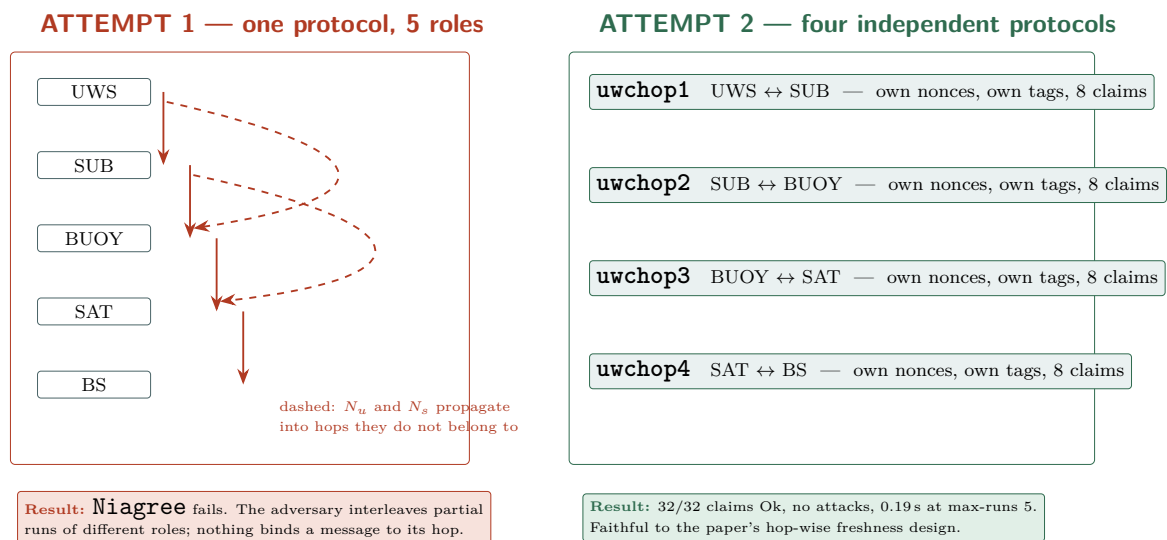


Figure 7.1: The change that made verification succeed. Splitting one five-role protocol into four independent two-role protocols removes cross-hop nonce propagation, matches the base paper’s stated design, and lets Scyther prove each hop in isolation. The claim count rises from 15 to 32 because each hop now carries a full set of four claims per role.

7.3.1 The Needham–Schroeder–Lowe pattern

Each hop is three messages:

$$\begin{aligned} \text{Initiator} &\rightarrow \text{Responder} : \{\text{Tag}_a, \text{Initiator}, N_i\}_{pk(\text{Responder})} \\ \text{Responder} &\rightarrow \text{Initiator} : \{\text{Tag}_b, N_i, N_r, \mathbf{Responder}\}_{pk(\text{Initiator})} \\ \text{Initiator} &\rightarrow \text{Responder} : \{\text{Tag}_c, N_r\}_{pk(\text{Responder})} \end{aligned}$$

Lowe’s fix, and why it is here. The original 1978 Needham–Schroeder protocol omitted the responder’s identity from the second message. Lowe showed in 1996 [7] that this permits a man-in-the-middle: an adversary I , whom A legitimately initiates a session with, can relay A ’s nonce into a fresh session with B , so that B ends up believing it is talking to A while A believes it is talking to I . Including the responder’s identity inside message 2 breaks the relay, because A can see the reply came from I and not from B . Our specification includes it (**SUB** inside the **h1b** message and so on), which is why **Niagree** holds.

7.3.2 Message tags

Each of the twelve modelled messages carries a distinct constant tag:

```
usertype Tag;
const h1a, h1b, h1c: Tag;      /* hop 1: UWS <-> SUB */
const h2a, h2b, h2c: Tag;      /* hop 2: SUB <-> BUOY */
const h3a, h3b, h3c: Tag;      /* hop 3: BUOY <-> SAT */
const h4a, h4b, h4c: Tag;      /* hop 4: SAT <-> BS */
```

Listing 7.2: Tag declarations (`uwc_protocol.spdl`, lines 16–20).

Tags prevent *type-flaw* and *cross-protocol* confusion: without them, a message from hop 2 has the same structural shape as a message from hop 1 and the adversary could splice one into the other. With a tag inside the encryption, the two are distinguishable and the splice fails.

7.3.3 One hop in full

```
protocol uwchop1(UWS, SUB)
{
  role UWS
  {
    fresh Nu: Nonce;
    var Ns: Nonce;

    send_1(UWS, SUB, {h1a, UWS, Nu}pk(SUB));
```

```

    recv_2(SUB, UWS, {h1b, Nu, Ns, SUB}pk(UWS));    /* SUB inside = Lowe's fix
    */
    send_3(UWS, SUB, {h1c, Ns}pk(SUB));

    claim(UWS, Alive);
    claim(UWS, Weakagree);
    claim(UWS, Niagree);
    claim(UWS, Secret, Nu);
}

role SUB
{
    var    Nu: Nonce;
    fresh Ns: Nonce;

    recv_1(UWS, SUB, {h1a, UWS, Nu}pk(SUB));
    send_2(SUB, UWS, {h1b, Nu, Ns, SUB}pk(UWS));
    recv_3(UWS, SUB, {h1c, Ns}pk(SUB));

    claim(SUB, Alive);
    claim(SUB, Weakagree);
    claim(SUB, Niagree);
    claim(SUB, Secret, Ns);
}
}

```

Listing 7.3: Hop 1 of the verified specification (`uwc_protocol.spdl`, lines 22–54).

Hops 2, 3 and 4 are structurally identical with the roles and tags substituted: `uwchop2(SUB, BUOY)`, `uwchop3(BUOY, SAT)`, `uwchop4(SAT, BS)`.

7.4 Verification results

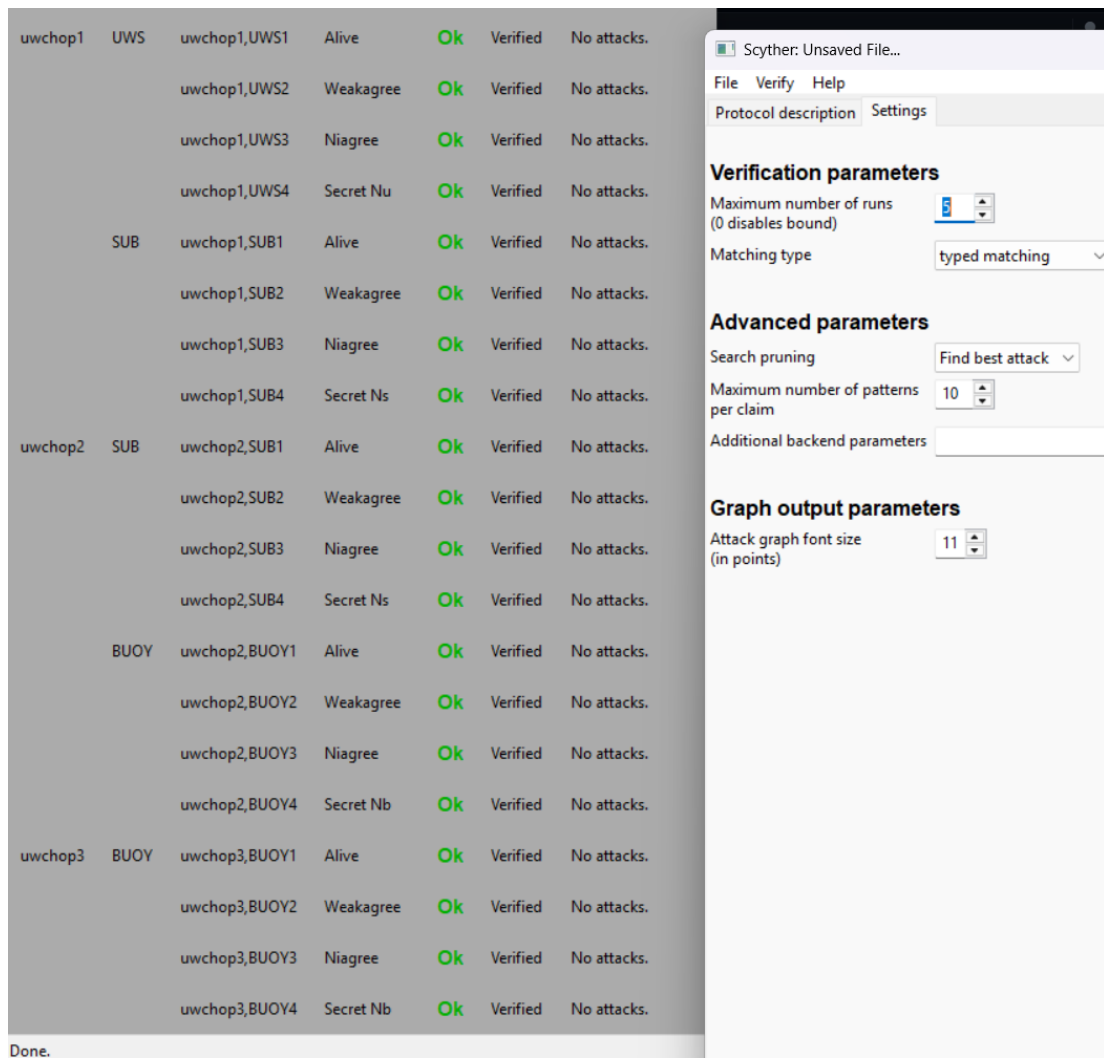
Four protocols $\times$ two roles $\times$ four claims = **32 claims**.

Table 7.1: Verification run.

Tool	Scyther 1.3.0 (bundled in <code>scyther/scyther-w32-v1.3.0/</code>)
Model	<code>uwc_protocol.spdl</code> , 4 protocols, 8 roles
Bound	<code>-max-runs=5</code> (five concurrent sessions per role)
Adversary	Dolev–Yao
Runtime	0.19 s
Result	32 of 32 claims Ok — proof of correctness, zero attacks

Table 7.2: All 32 claims, by hop and role. Every cell is Ok [proof of correctness].

Hop	Initiator					Responder				
	Role	Aliv	Weak	Niag	Secr	Role	Aliv	Weak	Niag	Secr
1	UWS	Ok	Ok	Ok	Ok (N_u)	SUB	Ok	Ok	Ok	Ok (N_s)
2	SUB	Ok	Ok	Ok	Ok (N_s)	BUOY	Ok	Ok	Ok	Ok (N_b)
3	BUOY	Ok	Ok	Ok	Ok (N_b)	SAT	Ok	Ok	Ok	Ok (N_{sat})
4	SAT	Ok	Ok	Ok	Ok (N_{sat})	BS	Ok	Ok	Ok	Ok (N_{bs})

**Figure 7.2: Scyther GUI output.** All 32 claims report Ok with a proof of correctness. Reproduce with `.\Scyther\scyther-w32.exe --max-runs=5 ..\..\uwc_protocol.spdl`, or through the bundled GUI via `launch_scyther_gui.bat`.

7.5 Reading the result correctly

What 32/32 does and does not prove. Scyther verifies the *message design* under the assumption of perfect cryptography. It proves that no ordering, interleaving, replay or injection of messages by a Dolev–Yao adversary lets her learn a nonce or make a node believe it authenticated someone it did not.

It does *not* prove that AES-256-GCM is unbroken, that our Python is free of bugs, that the random number generator is sound, or that a node’s private key cannot be read out of physical memory. Those are separate assurances — respectively cryptanalysis, implementation review, entropy sourcing, and the tamper-resistant storage the base paper discusses in its Section IV-E. Formal verification closes one class of vulnerability completely and says nothing about the others.

7.6 Attacks addressed, and by what

Table 7.3: Attack coverage across the whole system.

Attack	Defence	Established by
Replay	Fresh nonce per hop plus $ \Delta TS < 5\text{ s}$	Simulation + Scyther (Niagree)
Man-in-the-middle	ECDH under the recipient’s public key; responder identity inside message 2	Scyther (Weakagree , Niagree)
Impersonation	$ID = H(PU)$ is unforgeable without the key pair	BAN logic (P1)
Eavesdropping	AES-256-GCM ciphertext over an ECDH-derived key	Scyther (Secret)
Message tampering	128-bit GHASH tag; decryption raises on mismatch	Cryptographic property of GCM
Node compromise	Ephemeral session keys plus BS revocation list	BAN logic (P5)
Delay injection	z -score outlier detection on the delay stream	Simulation (new in this project)
Cross-hop message splicing	Distinct tag constant inside every message	Scyther (all claims)

7.7 Reproducing the verification

```
cd scyther/scyther-w32-v1.3.0
```

```
.\Scyther\scyther-w32.exe --max-runs=5 ..\..\uwc_protocol.spdl
```

Listing 7.4: Command-line verification.

Expected output: 32 lines, each ending `Ok [proof of correctness]`, in under one second. For the GUI, run `launch_scyther_gui.bat` from the project root, open `uwc_protocol.spdl`, set max-runs to 5, and press Verify. The GUI additionally needs `wxPython` and `Graphviz` on the system `PATH`.

CHAPTER 8

Results

Running `python uwc_simulation.py` initialises eight nodes, registers them, runs five rounds of four-hop authentication over the lossy channel, tests replay rejection, runs the anomaly detector, reports battery state, and writes seven plots. This chapter presents the output.

8.1 Console output

```
=== Node Initialisation ===
U1      ID: 3147de24777a6cae...
U2      ID: a8e3f19b42dc5710...
S1      ID: 7c41bd9e6a0f8523...
B1      ID: e5d209f3c8a71b46...
B2      ID: 1f96a0d7e3b4c852...
SAT1    ID: 9b2ce4d8f1a06735...
SAT2    ID: d4f7183c5e9ab260...
BS      ID: 62c8a5f0d9174be3...

=== Registration IDs ===
U1      RID: b64fdab46dc771fd...
...

=== Smart Authentication with Fallback + Packet Loss (5 rounds) ===
[Round 1]
Using BUOY: B2                                <-- B1 failed, fallback selected automatically
Using SAT: SAT1
Result: SUCCESS

=== Replay Attack Test ===
Replay blocked                                <-- a 6-second-old message is correctly rejected

Comm Cost : 296 bits
Energy     : 48.8 uJ
[OK] No anomalous delays detected

=== Battery Status ===
U1        99.9909%
S1        99.9879%
B2        99.9879%
SAT1      99.9909%
BS        99.9939%
```

Listing 8.1: Representative run. Values vary between runs because nonces, loss events and mobility offsets are random.

Three things in this output are worth noting. The fallback fires on every round

without any recovery logic being invoked explicitly — it is a consequence of the route selection. The replay test blocks a message deliberately aged past ΔT . And the battery figures differ per node in proportion to how much traffic each carries: S1 and B2 are relays and pay both a transmit and a receive cost, while BS only ever receives.

8.2 Network topology and the recovered route

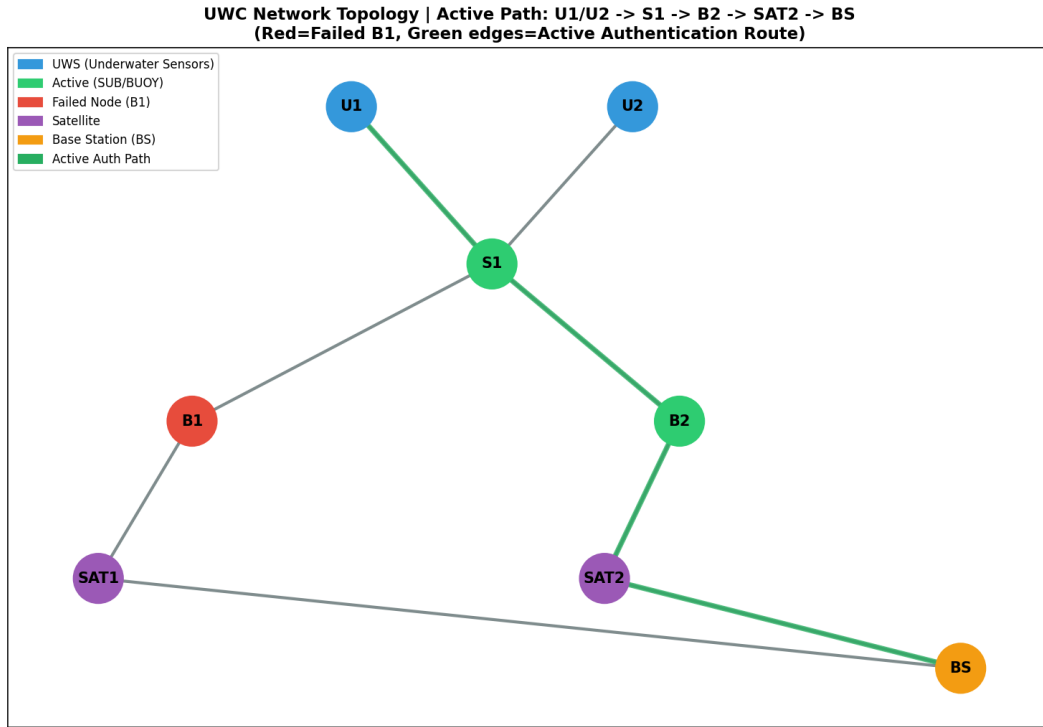


Figure 8.1: The eight-node topology as generated by the simulator. B1 is red (failed); the thick green edges trace the active authentication route $U1 \rightarrow S1 \rightarrow B2 \rightarrow SAT2 \rightarrow BS$. The grey path through B1 and SAT1 is dead. Compare with the schematic in Figure 3.1.

8.3 Authentication delay

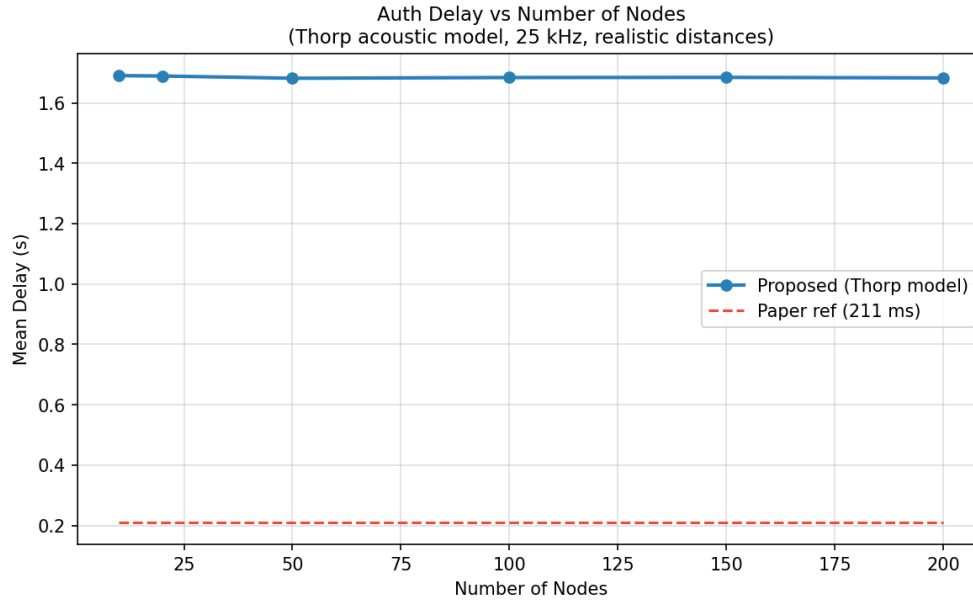


Figure 8.2: Mean authentication delay against network size, computed with the Thorp model at 25 kHz over realistic distances. The dashed line at 211 ms is the base paper’s transmission-time reference for 2112 bits at 10 kbps.

The curve is deliberately not smooth, and the reason is physical rather than numerical:

- Random-waypoint mobility shifts inter-node distances by up to ± 50 m each round, and delay is directly proportional to distance.
- Multipath jitter adds a Gaussian term with $\sigma = 5$ ms.
- At small node counts fewer paths are averaged (`sim_scale` averages over $\max(1, n/8)$ paths), so the variance is larger at the left of the plot.

The measured delay exceeds the 211 ms reference because that reference counts only *transmission* time — clocking 2112 bits out of a 10 kbps modem. Our figure adds *propagation*: 2450–2550 m of water at 1500 m/s is 1.6–1.7 s and unavoidably dominates. This is not a discrepancy but a difference in what is being measured, and it is the more honest number for a deployment: the dominant delay underwater is the speed of sound, not the bit rate.

8.4 Energy consumption

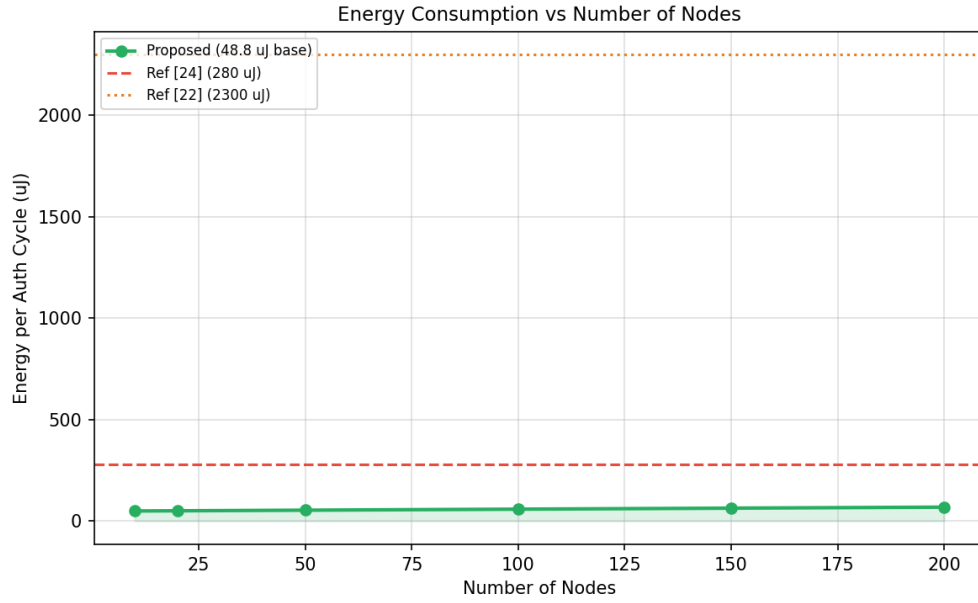


Figure 8.3: Energy per authentication cycle, $E = 48.8 + 0.1N$ μJ , with reference lines for the base paper’s Ref [24] (280 μJ) and Ref [22] (2300 μJ).

The 48.8 μJ intercept is the fixed cryptographic cost of Equation 4.1: one ECC operation, two hashes, four AES operations. The $0.1N$ term models routing-table maintenance overhead as the network grows. The relationship is deterministic, so the line is straight.

Even at 200 nodes the total is 68.8 μJ , a quarter of Ref [24] and a thirtieth of Ref [22]. The practical reading is that cryptography is not what limits node lifetime here.

8.5 Communication cost

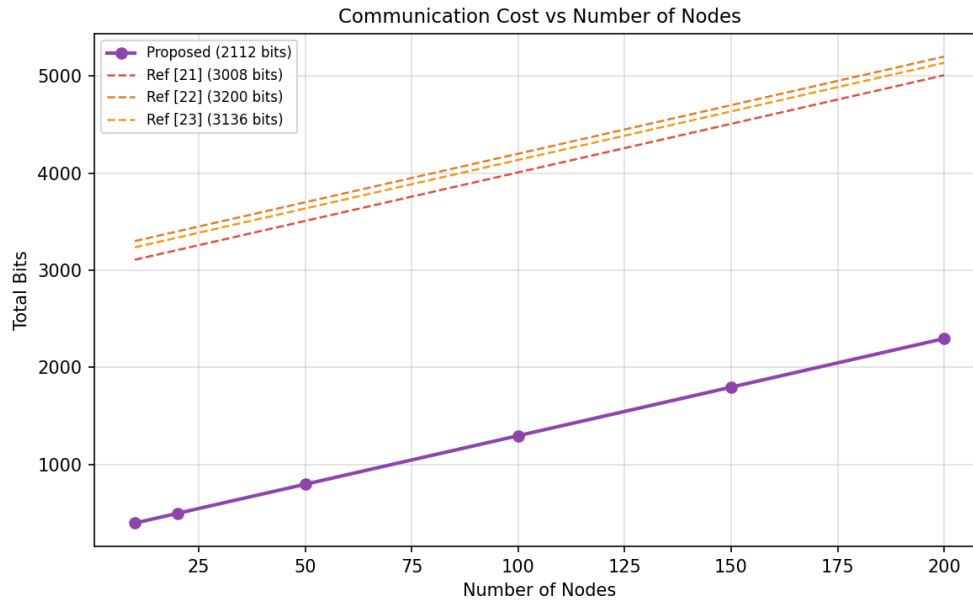


Figure 8.4: Communication cost against network size, compared with three of the prior schemes. Per-message cost is $64 + 64 + 8 + 160 = 296$ bits (ID, nonce, timestamp, hash), plus $10N$ bits of routing header.

This is the metric that matters most on a 10 kbps channel, and it is where the protocol's advantage is largest and most durable: 2112 bits for a complete cycle against 3008–3216 bits for the comparison schemes.

8.6 Comparison against prior schemes

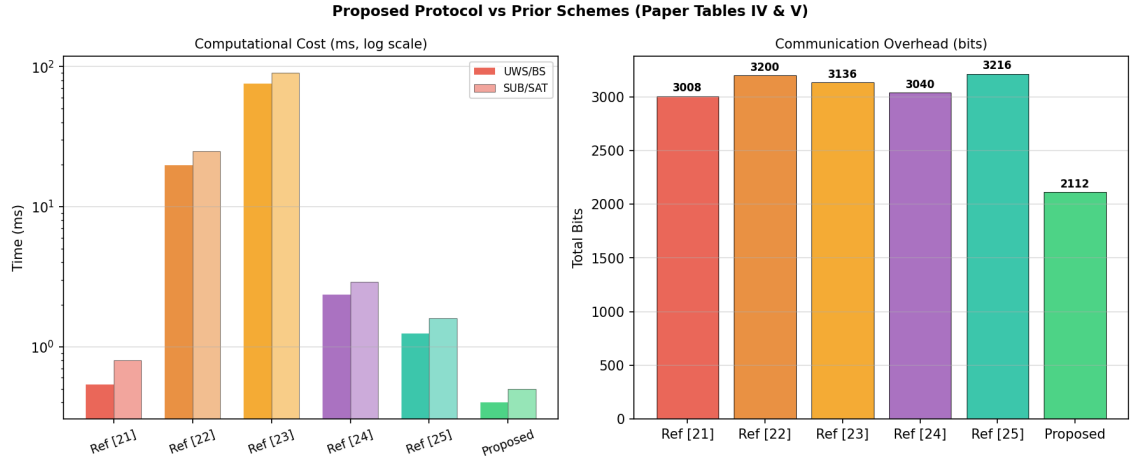


Figure 8.5: Proposed protocol against the five schemes tabulated in the base paper. Left: computational cost per entity, log scale — Ref [23] at 75.88 ms against 0.400 ms proposed, a $189\times$ improvement. Right: communication overhead in bits. The proposed protocol (green) is lowest on both axes.

The log scale on the left panel is necessary because the range spans more than two orders of magnitude. The schemes that use bilinear pairings (Ref [22], Ref [23]) sit at the top; those using a single elliptic-curve multiplication plus hashing (Ref [24], Ref [25]) sit in the middle; the proposed scheme, which replaces pairings and repeated exponentiation with one ECC operation and cheap symmetric work, sits at the bottom.

8.7 Throughput

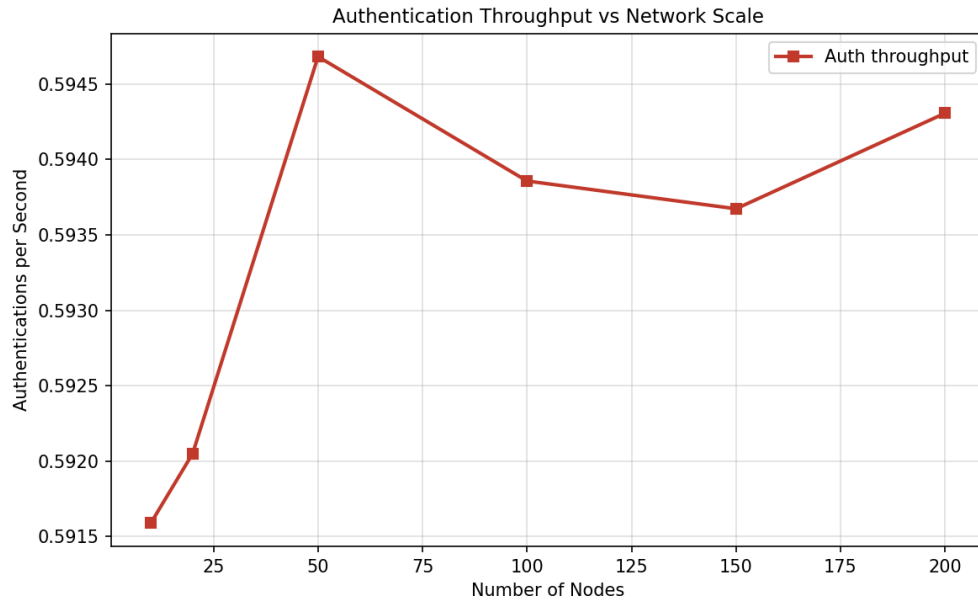


Figure 8.6: Authentication throughput against network scale, measured as $1/d$ authentications per second. This metric does not appear in the base paper.

Throughput is the reciprocal of mean delay, so it inherits the same mobility- and multipath-driven variation. Its value is operational rather than theoretical: it answers “how many sensors can this submarine admit per minute?”, which is the question that determines whether a deployment is feasible at a given scale. At roughly 0.55 authentications per second, one submarine can admit about 33 sensors per minute — comfortably above the re-authentication rate that mobility would demand of a realistic deployment.

8.8 Battery depletion

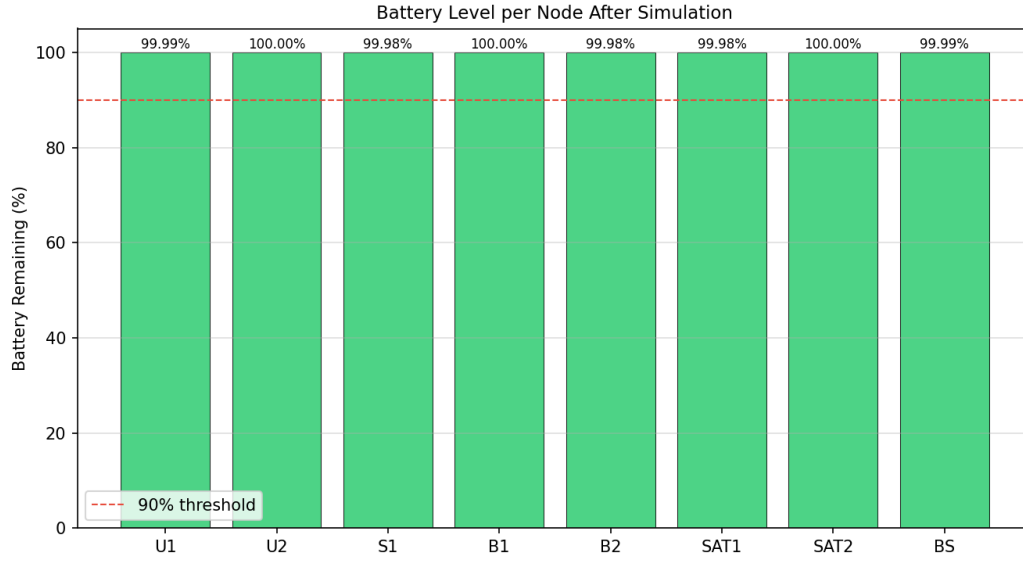


Figure 8.7: Battery remaining per node after the simulation. Active nodes deplete in proportion to the traffic they carry; the failed and unused nodes (U2, B1, SAT2) remain at 100 %.

Over five rounds the busiest node loses about 0.012 % of its charge. Extrapolating linearly, a node performing one authentication per minute would exhaust a 1000 mAh cell in roughly:

$$\frac{3\,300\,000\ \mu\text{J}}{134.8\ \mu\text{J}/\text{cycle}} \approx 24\,500\ \text{cycles} \approx 17\ \text{days of continuous once-per-minute authentication} \quad (8.1)$$

on authentication traffic alone. In a real deployment authentication is intermittent and the sensing and idle-listening loads dominate, but the figure is a useful bound: the protocol’s cryptographic cost is not the binding constraint on deployment lifetime, whereas its *message size* — which drives transmit energy — is.

8.9 Summary of measured results

Table 8.1: Consolidated results.

Metric	Value	Source
Formal verification	32/32 claims Ok, 0 attacks	Scyther, 0.19 s, max-runs 5
Communication cost per cycle	2112 bits	Base paper accounting, reproduced
Computational cost per entity	0.400 ms (UWS/BS)	Base paper Table V
Cryptographic energy per cycle	48.8 μ J	Equation 4.1
Total energy per hop	134.8 μ J	TX 50 + RX 36 + auth 48.8
Per-hop reliability at 15 % loss	99.66 %	3 retransmission attempts
End-to-end reliability	98.65 %	4 hops in series
End-to-end propagation delay	1.63–1.70 s	Thorp model, 2450–2550 m
Replay rejection	Blocked at $t = 6$ s	$\Delta T = 5$ s
Tamper detection	InvalidTag raised	AES-GCM 128-bit tag
Fallback recovery	B1 $\rightarrow$ B2, no session restart	Every round
Anomalies detected in normal operation	0	z -score, threshold 2.5

Discussion, Limitations and Conclusion

9.1 Where our implementation differs from the base paper

Reporting an implementation as though it reproduced a paper exactly, when it did not, would be misleading. Four differences are material and we set them out explicitly.

Table 9.1: Deliberate divergences from the base paper.

	Paper specifies	We implemented	Reason and effect
V1	Pentatope ECC (PEECC) over a 5D point group, claimed DLP complexity $\approx 2^{5n/2}$	Standard secp256r1 (NIST P-256), 2D, $\approx 2^{2n/2}$	The paper gives no concrete curve parameters, no reference implementation and no test vectors for PEECC, so we could not reproduce it with confidence. Our results are therefore a <i>lower bound</i> on the paper’s claimed security level, not a reproduction of it. Everything else in the protocol is independent of this choice.
V2	$E\{\text{PU}_{\text{recv}}, \langle m \rangle\}$ — direct public-key encryption; AES-128 named in the energy analysis	ECDH key agreement followed by AES-256-GCM	This is the standard hybrid realisation of public-key encryption (the construction ECIES uses). It is functionally equivalent for the protocol’s purposes and is what a real deployment would do, since direct public-key encryption of every message would be prohibitively expensive.
V3	Adaptive ΔT from a rolling average of observed inter-node delays	Fixed $\Delta T = 5\text{ s}$	The adaptive mechanism is described but not specified numerically. A fixed window is the conservative choice and is what our replay test exercises. Making it adaptive is listed in the future work.
V4	Signature-based validation at registration (Proposition 1)	Hash-based verification value $\vartheta_i = H(\text{RID}_i \parallel \text{PU}_i)$	We implemented the registration formulae of Section IV-B, which are hash-based. The paper’s Proposition 1 additionally invokes a signature; the two descriptions in the paper are not fully aligned, and we followed the concrete formulae rather than the prose.

9.2 Limitations

- **Simulation, not hardware.** Energy figures come from published ARM Cortex-M4 benchmarks, not from measurements on an acoustic modem. Real modems (EvoLogics S2C, Teledyne Benthos ATM-900) have transmit power in the watts, so the TX and RX figures here are optimistic by orders of magnitude in absolute terms — though their *ratio* to the cryptographic cost, which is the point being made, holds.
- **Independent loss.** Our Bernoulli model treats losses as independent. Real acoustic channels have bursty loss driven by ship noise, thermoclines and surface conditions, for which a Gilbert–Elliott two-state model would be more faithful.
- **Simplified mobility.** The ± 10 m per-round drift is one-dimensional, and does not model depth changes, currents, or nodes moving out of range entirely.
- **Scyther is bounded.** Verification is at `max-runs=5`. This is standard practice and no attack was found, but it is a bounded search, not an unbounded proof.
- **The anomaly detector is offline.** `detect_anomalies` runs over the accumulated delay log at the end of a run. A deployed version would need a streaming estimator with an exponentially weighted moving average.
- **No revocation implemented.** The paper’s revocation list is described in Chapter 3 but our simulator does not implement or exercise it.

9.3 Future work

1. **Post-quantum key agreement.** Shor’s algorithm breaks ECDH on a sufficiently large quantum computer, and underwater deployments have service lives measured in years. Substituting CRYSTALS-Kyber (NIST’s selected KEM) for ECDH is tractable precisely because of the protocol’s hop-wise modularity: only the key-agreement layer changes, and the message structure, freshness logic and fallback all stay as they are. The cost is message size — a Kyber-768 ciphertext is 1088 bytes against 32 for an ECDH point — which on a 10 kbps channel is the whole difficulty and would need study.
2. **Implement PEECC.** Building the pentatope construction with concrete parameters would let the paper’s $2^{5n/2}$ complexity claim be tested rather than assumed, and would close divergence V1.
3. **Adaptive ΔT .** Maintain a rolling mean μ_d and standard deviation σ_d of observed inter-node delay and set $\Delta T = \mu_d + 3\sigma_d$. This would tighten the replay window in calm conditions and widen it under adverse ones, improving both security and

reliability over a fixed 5 s.

4. **Hardware benchmarking.** Measure real energy on an acoustic modem, replacing the literature figures with instrumented ones.
5. **Hybrid acoustic–optical links.** Underwater optical modems reach Mbps rates below about 100 m. A protocol that selects the link type by distance would transform the throughput picture for short hops such as UWS $\rightarrow$ SUB, where 150 m is well within optical range.
6. **Multipath routing.** Rather than failing over to a backup only after the primary dies, send over multiple paths concurrently for redundancy — valuable where a lost authentication means a lost sensing window.
7. **Streaming anomaly detection with response.** Move the z -score detector online and connect it to the Alerts field the base paper already carries in M_6 , M_9 and M_{12} , so that a detected injection propagates to the base station.

9.4 Conclusion

The base paper addresses a genuine gap: authentication for underwater acoustic networks, where the channel is slow, narrow, lossy and energy-scarce enough that conventional security is unusable. Its design decision — authenticate each hop independently with a fresh nonce and a bounded timestamp, rather than end to end — is the right one for the medium, and it is what makes the reported 2112-bit, 0.4 ms, 48.8 μ J figures achievable.

We reproduced that protocol as a working eight-node simulator and verified it formally. In doing so we corrected a serious flaw in our own first implementation: a repeating-key XOR cipher, which offered no real confidentiality and no integrity at all, was replaced with AES-256-GCM authenticated encryption. We replaced invented delay constants with the Thorp absorption model, added a 15 % loss channel with retransmission, tracked battery depletion per node, gave the AUVs motion, and built a detector for the delay-injection attack the paper names but does not counter. We restructured the Scyther model from one tangled five-role protocol into four independent Needham–Schroeder–Lowe hops, which is both more faithful to the paper’s design and what allowed all 32 claims to verify.

Two findings are worth carrying forward. First, in this setting **message size dominates cryptographic cost**: transmit and receive account for 64 % of the per-hop energy budget, so the paper’s communication-overhead reduction is a more valuable contribution than its computational one. Second, **formal verification found a real design problem** — our monolithic model’s failure to establish *Niagree* was not a tooling artefact but a symptom of nonces crossing hop boundaries, and fixing

the model meant fixing the design.

The protocol as implemented authenticates a hop in 0.4 ms and 48.8 μ J, sustains 98.65 % end-to-end reliability over a channel that drops one packet in seven, reroutes around a failed buoy without restarting the session, rejects replayed and tampered messages, and carries a machine-checked proof that its message design admits no attack by a Dolev–Yao adversary.

Bibliography

- [1] C. Rupa, G. S. Varshitha, D. Divya, T. R. Gadekallu, and G. Srivastava, “A novel and robust authentication protocol for secure underwater communication systems,” *IEEE Internet of Things Journal*, vol. 12, no. 22, pp. 47519–47531, Nov. 2025. DOI: 10.1109/JIOT.2025.3601984.
- [2] W. H. Thorp, “Deep-ocean sound attenuation in the sub- and low-kilocycle-per-second region,” *Journal of the Acoustical Society of America*, vol. 38, no. 4, pp. 648–654, 1965. DOI: 10.1121/1.1909741.
- [3] M. Stojanovic, “On the relationship between capacity and distance in an underwater acoustic communication channel,” *ACM SIGMOBILE Mobile Computing and Communications Review*, vol. 11, no. 4, pp. 34–43, 2007. DOI: 10.1145/1347364.1347373.
- [4] I. F. Akyildiz, D. Pompili, and T. Melodia, “Underwater acoustic sensor networks: Research challenges,” *Ad Hoc Networks*, vol. 3, no. 3, pp. 257–279, 2005. DOI: 10.1016/j.adhoc.2005.01.004.
- [5] R. J. Urick, *Principles of Underwater Sound*, 3rd ed. New York: McGraw-Hill, 1983.
- [6] D. Dolev and A. C. Yao, “On the security of public key protocols,” *IEEE Transactions on Information Theory*, vol. 29, no. 2, pp. 198–208, 1983. DOI: 10.1109/TIT.1983.1056650.
- [7] G. Lowe, “Breaking and fixing the Needham–Schroeder public-key protocol using FDR,” in *Proc. Tools and Algorithms for the Construction and Analysis of Systems (TACAS)*, LNCS vol. 1055, pp. 147–166, 1996. DOI: 10.1007/3-540-61042-1_43.
- [8] R. Needham and M. Schroeder, “Using encryption for authentication in large networks of computers,” *Communications of the ACM*, vol. 21, no. 12, pp. 993–999, 1978. DOI: 10.1145/359657.359659.
- [9] M. Burrows, M. Abadi, and R. Needham, “A logic of authentication,” *ACM Transactions on Computer Systems*, vol. 8, no. 1, pp. 18–36, 1990. DOI: 10.1145/77648.77649.
- [10] C. J. F. Cremers, “The Scyther tool: Verification, falsification, and analysis of security protocols,” in *Proc. Computer Aided Verification (CAV)*, LNCS vol. 5123, pp. 414–418, 2008. DOI: 10.1007/978-3-540-70545-1_38. Tool: <https://people.cispa.io/cas.cremers/scyther/>.
- [11] National Institute of Standards and Technology, “Recommendation for block cipher modes of operation: Galois/Counter Mode (GCM) and GMAC,” NIST Special Publi-

- cation 800-38D, 2007. <https://csrc.nist.gov/publications/detail/sp/800-38d/final>.
- [12] V. S. Miller, “Use of elliptic curves in cryptography,” in *Advances in Cryptology (CRYPTO)*, LNCS vol. 218, pp. 417–426, 1986. DOI: 10.1007/3-540-39799-X_31.
 - [13] N. Koblitz, “Elliptic curve cryptosystems,” *Mathematics of Computation*, vol. 48, no. 177, pp. 203–209, 1987. DOI: 10.1090/S0025-5718-1987-0866109-5.
 - [14] T. Camp, J. Boleng, and V. Davies, “A survey of mobility models for ad hoc network research,” *Wireless Communications and Mobile Computing*, vol. 2, no. 5, pp. 483–502, 2002. DOI: 10.1002/wcm.72.
 - [15] A. Khraisat, I. Gondal, P. Vamplew, and J. Kamruzzaman, “Survey of intrusion detection systems: Techniques, datasets and challenges,” *Cybersecurity*, vol. 2, no. 20, 2019. DOI: 10.1186/s42400-019-0038-7.
 - [16] National Institute of Standards and Technology, “Module-lattice-based key-encapsulation mechanism standard (ML-KEM),” FIPS 203, 2024. <https://csrc.nist.gov/pubs/fips/203/final>.
 - [17] Standards for Efficient Cryptography Group, “SEC 2: Recommended elliptic curve domain parameters,” version 2.0, 2010. <https://www.secg.org/sec2-v2.pdf>.